

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных средств

К защите допустить:
Заведующий кафедрой ЭВС
_____ И.С. Азаров

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к дипломному проекту
на тему

**IP-ЯДРО НЕЙРОННОЙ СЕТИ ПРЯМОГО РАСПРОСТРАНЕНИЯ ДЛЯ
РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР**

БГУИР ДП 1-40 02 02 01 014 ПЗ

Студент	Е.А. Кривальцевич
Руководитель	М.И. Вашкевич
Консультанты: <i>от кафедры ЭВС</i>	М.И. Вашкевич
<i>по экономическому разделу</i>	И.В. Смирнов
Нормоконтролёр	Д.С. Лихачёв
Рецензент	

Минск 2025

СОДЕРЖАНИЕ

Введение.....	6
1 Обзор существующих нейронных сетей для распознавания рукописных цифр	7
1.1 Нейронные сети	7
1.2 Нейронные сети для распознавания рукописных цифр	7
1.3 Полносвязные нейронные сети	8
1.4 Сверточные нейронные сети	9
2 Разработка нейронной сети для распознавания рукописных цифр	10
2.1 Принцип работы LST преобразования	10
2.2 Обучение нейронной сети	12
3 Разработка структуры IP-блока нейронной сети для распознавания рукописных цифр	14
3.1 Описание структурной схемы	14
3.2 Программная реализация эталонной модели нейронной сети на языке Python	16
3.3 Разработка операционной части нейронной сети	18
3.4 Проектирование функциональной схемы нейронной сети	20
3.5 Построение алгоритма работы нейронной сети	21
3.6 Проектирование управляющей части	21
3.7 Построение графа переходов управляющего автомата	23
4 Аппаратная реализация IP-блока нейронной сети для распознавания рукописных цифр	26
4.1 Описание пакета Xilinx Vivado	26
4.2 Описание функциональных блоков	27
4.3 Описание интерфейса нейронной сети	28
4.4 Описание процесса создания блочной диаграммы	28
4.5 Результаты синтеза и симуляции	31
5 Анализ результатов тестирования IP-блока нейронной сети для распознавания рукописных цифр	36
5.1 Описание среды тестирования	36
5.2 Тестирование разработанного IP-блока нейронной сети	37
5.3 Экспериментальное исследование IP-блока нейронной сети	38
5.4 Сравнение с аналогичными архитектурами	41
Заключение	43
Список использованных источников	44

ВВЕДЕНИЕ

В последние годы значительный интерес вызывает аппаратная реализация нейронных сетей, поскольку программные решения, работающие на центральных процессорах (CPU) и графических процессорах (GPU), не всегда обеспечивают требуемую скорость обработки и энергоэффективность.

В связи с этим использование специализированных аппаратных ускорителей на базе FPGA (Field-Programmable Gate Array) или ASIC (Application-Specific Integrated Circuit) становится актуальной задачей. Аппаратная реализация нейронной сети в виде IP-ядра позволяет значительно повысить производительность и снизить задержки обработки данных, что особенно важно для встроенных систем.

При реализации нейронных сетей на базе CPU и GPU как правило используются стандартизированные типы данных. При реализации на базе FPGA появляется возможность использовать для представления параметров нейронных сетей типов данных, обеспечивающих различную точность. Причем выбор точности представления напрямую будет влиять на аппаратные затраты[1].

Целью данного проекта является разработка IP-ядра нейронной сети прямого распространения для распознавания рукописных цифр. Необходимо не только создать высокоэффективное аппаратное решение, но и в продемонстрировать его практической применимости в условиях ограниченных вычислительных ресурсов. Разработанное IP-ядро может быть использовано в системах реального времени, таких как интеллектуальные камеры, портативные устройства и робототехнические комплексы, где требуется автономная и быстрая обработка визуальной информации без привлечения внешней вычислительной инфраструктуры.

В данном проекте используется обученное двумерное разделяемое преобразование (LST2D), которое рассматривается как новый тип слоя нейронной сети. Он следует концепции распределения веса.

1 ОБЗОР СУЩЕСТВУЮЩИХ НЕЙРОННЫХ СЕТЕЙ ДЛЯ РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР

1.1 Нейронные сети

Нейронная сеть представляет из себя совокупность нейронов, соединенных друг с другом определенным образом. Нейрон представляет из себя элемент, который вычисляет выходной сигнал из совокупности входных сигналов. Структура нейрона представлена на рисунке 1.1.

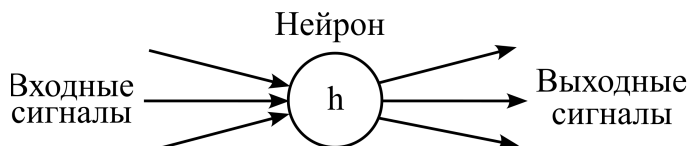


Рисунок 1.1 – Общее представление нейрона

Нейроны выполняют следующую последовательность действий:

- приём входных сигналов;
- комбинирование входных сигналов;
- вычисление выходных сигналов;
- передача выходных сигналов следующим нейронам.

Между собой нейроны могут быть соединены абсолютно по-разному, это определяется структурой конкретной сети. Искусственные нейронные сети можно рассматривать как направленный граф со взвешенными связями, в котором искусственные нейроны являются узлами. По архитектуре связей они могут быть сгруппированы в два класса: сети прямого распространения, в которых графы не имеют петель, и рекуррентные сети (RNN), или сети с обратными связями.

1.2 Нейронные сети для распознавания рукописных цифр

Распознавание рукописных цифр является одной из классических задач машинного обучения, для решения которой применяется широкий спектр нейронных сетей. В зависимости от сложности задачи, требований к точности, скорости обработки и аппаратных ограничений используются различные архитектуры, включая полносвязные сети (FCNN), сверточные нейронные сети (CNN) и более сложные гибридные модели.

Одним из первых методов распознавания рукописных цифр стали многослойные перцептроны (MLP), относящиеся к классу полносвязных нейронных сетей. Такие сети состоят из входного слоя, нескольких скрытых слоев и выходного слоя, где каждый нейрон соединен со всеми нейронами предыдущего и последующего слоев. Они способны успешно решать задачу распознавания, но требуют большого количества параметров и вычислительных ресурсов, что делает их менее эффективными.

Более эффективными оказались сверточные нейронные сети. Эти сети включают сверточные слои, которые извлекают характерные признаки из изображения. CNN значительно превосходят полносвязные сети в точности и скорости работы, поскольку используют пространственные зависимости в данных и требуют меньше параметров.

Хотя рекуррентные нейронные сети и их усовершенствованные версии, такие как Long Short-Term Memory Network (LSTM) и Gated Recurrent Unit Network (GRU), чаще применяются для обработки последовательных данных, они могут использоваться и для распознавания рукописных цифр. Они эффективны в случаях, когда рукописные символы анализируются в контексте строки.

Для повышения точности и эффективности могут применяться гибридные подходы, комбинирующие преимущества разных типов нейронных сетей. Например, модели, совмещающие CNN и LSTM, успешно применяются для распознавания рукописного текста, где CNN используется для выделения признаков, а LSTM – для анализа последовательностей. Рассмотрим подробнее полносвязные и сверточные нейронные сети.

1.3 Полносвязные нейронные сети

Полносвязная нейронная сеть – это базовая архитектура искусственных нейронных сетей, в которой каждый нейрон одного слоя соединен со всеми нейронами следующего слоя. Такая архитектура чаще всего применяется в качестве классификатора после сверточных или рекуррентных слоев. Количество нейронов на каждом слое может отличаться от соседних слоев.

Общая структура полносвязных нейронных сетей показана на рисунке 1.2.

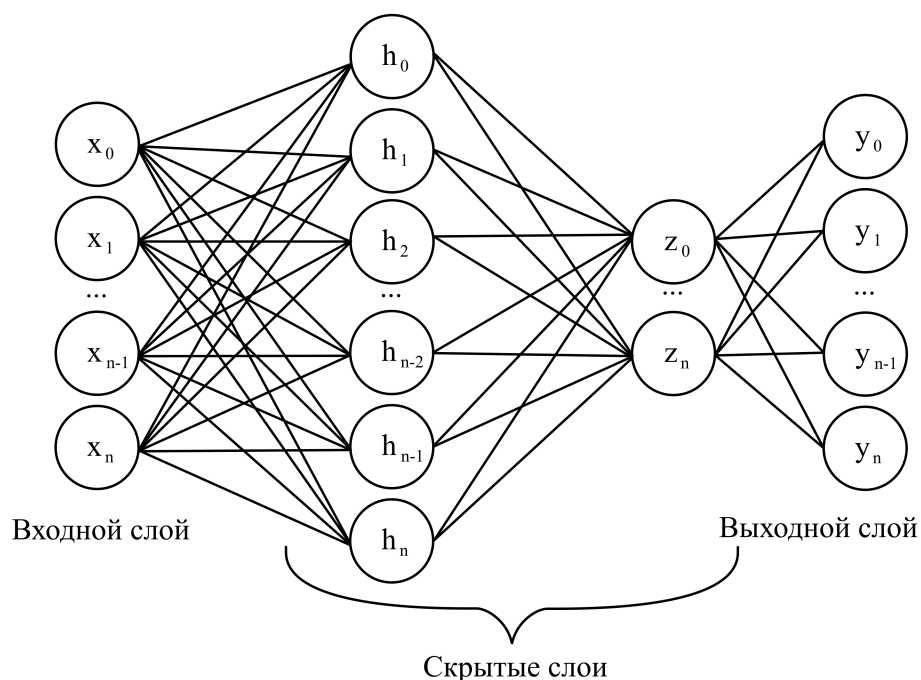


Рисунок 1.2 – Общая структура полносвязных нейронных сетей

Математически данную сеть можно описать формулой для вычисления выхода нейрона в полносвязном слое

$$h = f(Wx + b), \quad (1.1)$$

где x – входной вектор; W – матрица весов; b – вектор смещений; $f(\cdot)$ – функция активации; h – выходной вектор нейронов.

1.4 Сверточные нейронные сети

Сверточные нейронные сети (Convolutional Neural Networks) представляют собой архитектуру глубокого обучения, предназначенную для обработки данных обладающих сетчатой топологией, таких как изображения. Их основное преимущество заключается в способности эффективно извлекать пространственные и иерархические особенности входных данных с помощью операций свертки.

Принцип работы сверточных нейронных сетей показан на рисунке 1.3[3].

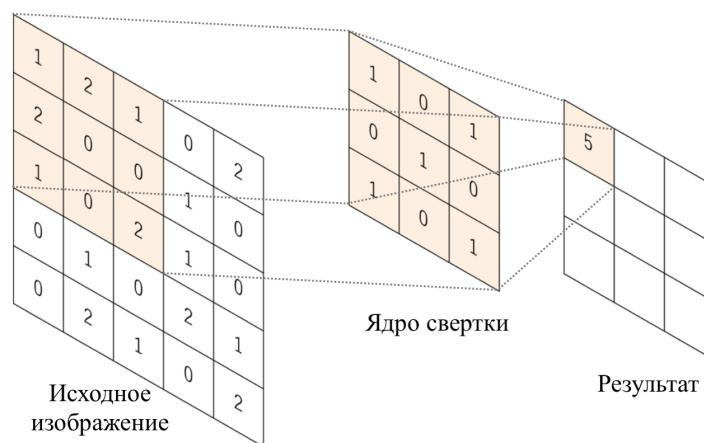


Рисунок 1.3 – Принцип работы сверточных нейронных сетей

Основная операция в сверточных нейронных сетях – это свертка, которая заключается в пошаговом применении фильтра (ядра свёртки) к локальным областям входного изображения:

$$S(i, j) = \sum_m \sum_n X(i + m, j + n) \cdot K(m, n), \quad (1.2)$$

где $X(i, j)$ – входное изображение; $K(m, n)$ – ядро свертки; $S(i, j)$ – выходное изображение после свертки.

Сверточный слой применяется к входному изображению, используя несколько фильтров, каждый из которых извлекает определенные характеристики. Как правило, чем глубже слой, тем более абстрактные признаки он способен захватывать.

2 РАЗРАБОТКА НЕЙРОННОЙ СЕТИ ДЛЯ РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР

2.1 Принцип работы LST преобразования

Двумерное разделимое преобразование используется в обработке изображений для уменьшения вычислительной сложности. Основная идея заключается в том, что вместо хранения полного двумерного ядра свёртки размером $n \times n$ используется его представление в виде произведения двух одномерных векторов. Это позволяет сократить число параметров с n^2 до $2n$, что снижает затраты на вычисления и память.

$$W = V \times h^T, \quad (2.1)$$

где $W \in \mathbb{R}^{n \times n}$; $v, h^T \in \mathbb{R}^{n \times 1}$.

LST основан на идее совместного использования весов одного полносвязного слоя для обработки всех строк изображения. После этого второй общий полносвязный слой используется для обработки всех столбцов представления изображения, полученных из первого слоя. Использование слоев LST в архитектуре нейронной сети значительно сокращает количество параметров модели ($N = 2 \cdot (d_{in} + 1) \cdot d_{out}$) по сравнению с моделями, которые используют стекированные полносвязные слои.

Принцип работы LST2D представлен на рисунке 2.1[8].

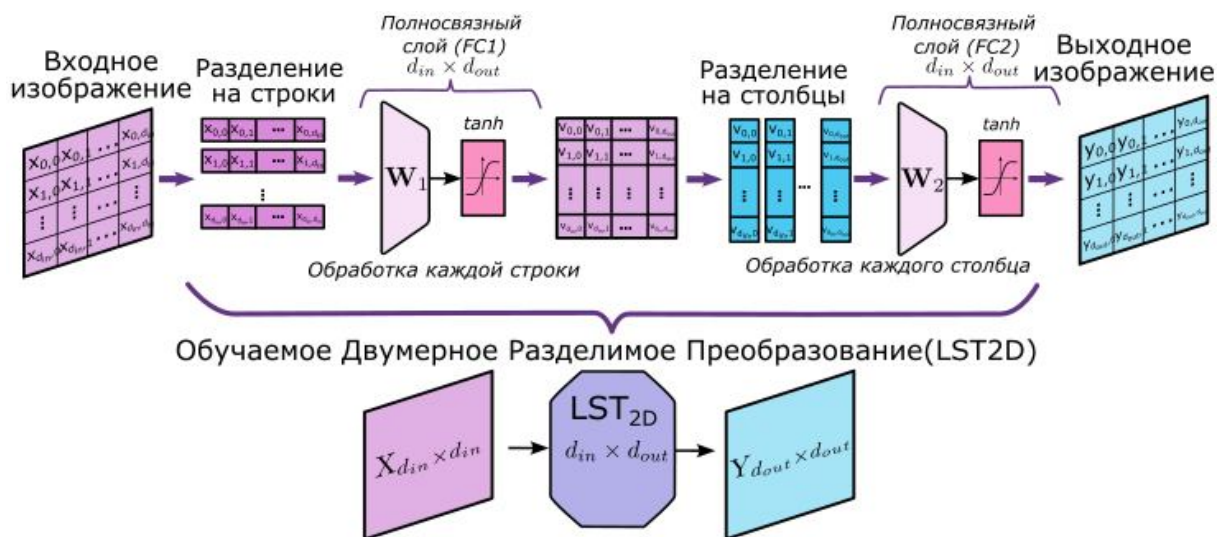


Рисунок 2.1 – Принцип работы LST2D

С математической точки зрения LST2D принимает двумерное изображение X размером $d_{in} \times d_{in}$ в качестве входных данных и создает двумерное выходное изображение Y размером $d_{out} \times d_{out}$.

$$Y = LST_{d_{in} \times d_{out}}(X) = \tan(W_2 \tan(W_1 X^T)). \quad (2.2)$$

где W_1 , W_2 — матрицы весов слоев FC1 и FC2 соответственно.

Алгоритм работы LST слоя, показывающий последовательность действий по преобразованию входного изображения размером 28×28 в выходное, такого же размера представлен на рисунке 2.2.

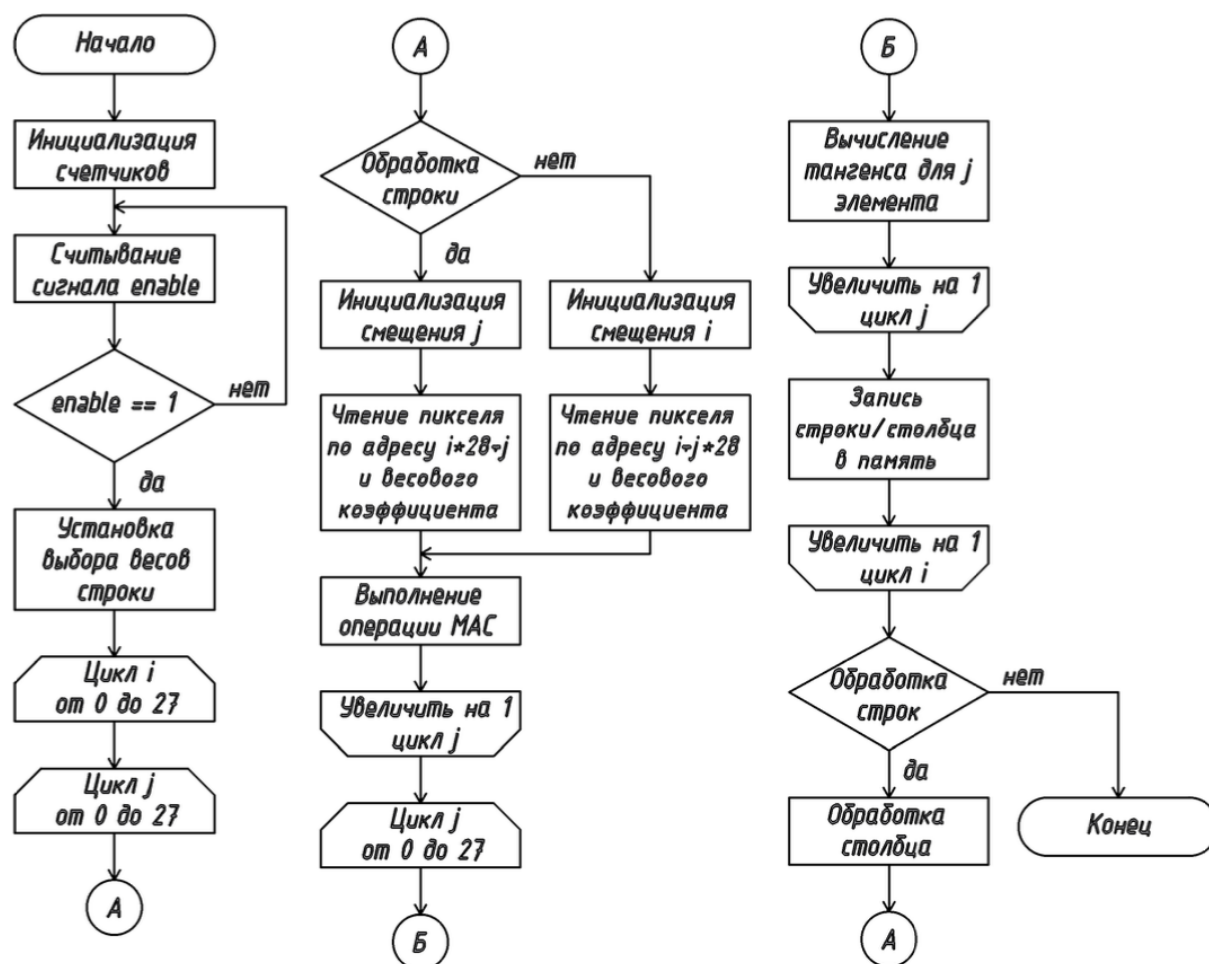


Рисунок 2.2 – Алгоритм работы LST2D слоя

Простейшая архитектура сети на основе LST2D показана на рисунке 2.3[8].

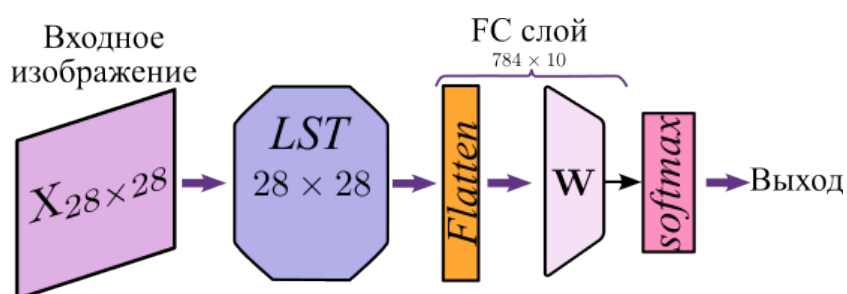


Рисунок 2.3 – Принцип работы LST-1

Данную архитектуру можно масштабировать(рисунок 2.4[8]).

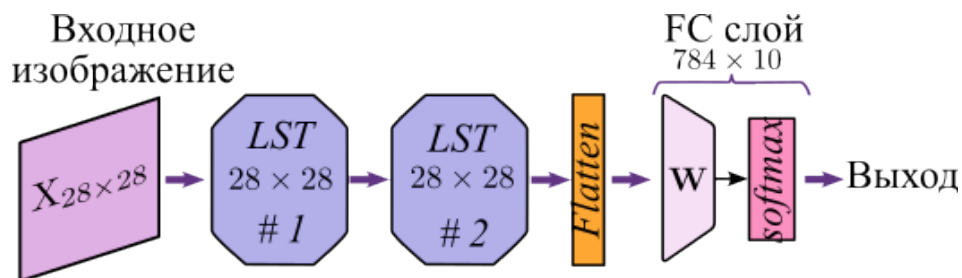


Рисунок 2.4 – Принцип работы LST-2

Пример обработки изображения нейронной сетью архитектуры LST-1 представлен на рисунке 2.5[8].

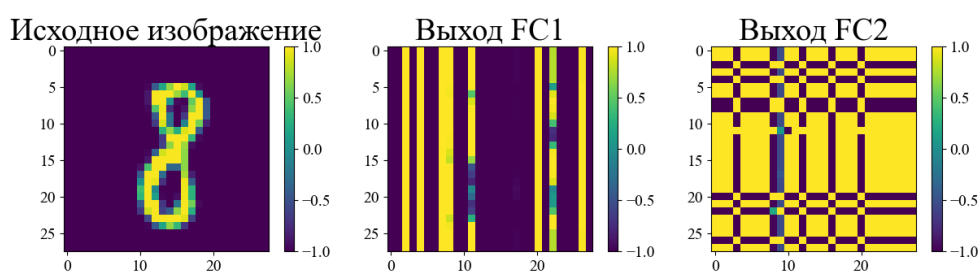


Рисунок 2.5 – Архитектура CNN-Transformer

2.2 Обучение нейронной сети

Обучение нейронной сети LST-1 на основе 60000 тренировочных изображений из базы данных MNIST. Для обучения используется фреймворк PyTorch.

В качестве исходных данных используется набор MNIST, содержащий 70 тысяч полутоновых изображений размера 28×28 пикселей, представляющих рукописные цифры от 0 до 9 [10]. Данный набор разделен на две части: тренировочная выборка – 60 тысяч изображений, тестовая выборка – 10 тысяч изображений.

Пример данных из базы MNIST представлен на рисунке 2.6.



Рисунок 2.6 – Пример данных из MNIST

Из тренировочного набора выделяется 1000 изображений для валидации, оставшиеся 59000 используются для непосредственного обучения модели.

Перед подачей в нейросеть изображения проходят предварительную обработку, включающую нормализацию значений пикселей. Для упрощения процесса

обучения выполняется нормализация данных таким образом, чтобы значение каждого пикселя было в диапазоне от -1 до 1 , а стандартное отклонение (σ) составляло 0.5 . Это позволяет сделать градиентный спуск более устойчивым и ускорить процесс сходимости модели. Пример такой обработки представлен на рисунке 2.7.

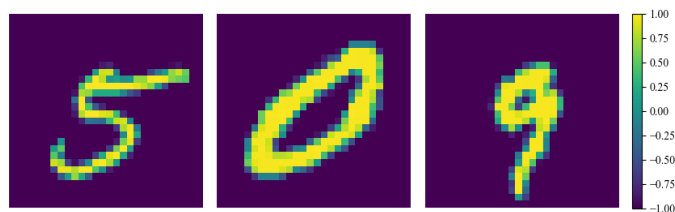


Рисунок 2.7 – Пример обработанных данных из MNIST

Архитектура модели L2DST представляет собой последовательное соединение двух линейных слоёв с функцией активации гиперболический тангенс $\tanh$ и одного полносвязного слоя с функцией активации softmax , применяемой для выполнения классификации. Основной особенностью реализации является использование слоёв LazyLinear, позволяющих автоматически определить размер входного пространства на этапе первого запуска модели.

Обучение модели осуществляется с использованием стохастического оптимизатора Adam, известного своей эффективностью на широком спектре задач глубокого обучения. Начальное значение скорости обучения задано равным 1.5×10^{-3} . Для обеспечения устойчивого процесса обучения применяется поэтапное снижение скорости обучения с помощью механизма StepLR. Каждые 10 эпох происходит уменьшение текущей скорости обучения на 3%, что позволяет модели более точно подстраиваться к минимальному значению функции потерь на поздних этапах обучения.

На рисунке 2.8 показан график функции потерь за 370 эпохах.

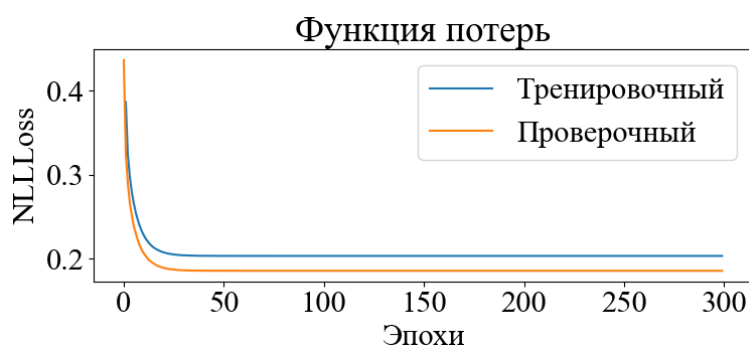


Рисунок 2.8 – Графики функции потерь на обучающем и тестовом наборе

После завершения каждой эпохи вычисляются средние значения ошибки на тренировочном и валидационном наборах данных. Python описание процесса обучения нейронной сети прямого распространения приведено в приложении Б.

3 РАЗРАБОТКА СТРУКТУРЫ IP-БЛОКА НЕЙРОННОЙ СЕТИ ДЛЯ РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР

3.1 Описание структурной схемы

Принцип работы данной нейронной сети основан на комбинации обучаемого двумерного разделимого преобразования и полносвязной нейронной сети. Такая архитектура позволяет эффективно обрабатывать изображения, снижая вычислительные затраты при аппаратной реализации.

Для окончательной классификации используется полносвязный слой с функцией активации softmax. Данный слой преобразует выходные значения в вероятностное распределение по классам, позволяя определить, к какой цифре относится входное изображение.

Вычислительное ядро разработанного устройства состоит из 28 МАС (Multiply-ACcumulate) ядер, распределенных по специализированным блокам. Эти блоки организованы таким образом, чтобы обеспечить эффективное выполнение операций умножения вектора изображения на матрицу весов.

Десять МАС-ядер из общего массива используются на финальном тапе вычислений и непосредственно участвуют в формировании входных данных для функции активации softmax. Эти ядра получают входные данные из трех независимых блоков памяти, что позволяет оптимизировать процесс выборки весов и повысить скорость вычислений. Остальные 18 МАС-ядер распределены по блокам, где каждая вычислительная единица получает данные из двух запоминающих устройств.

IP-блок устройства предназначен для интеграции в систему на кристалле (СнК) и взаимодействует с процессорной системой через AXI-uP интерфейсы. СнК – это интегральная схема, которая объединяет в одном чипе все основные компоненты вычислительной системы: процессор, ОЗУ и ПЗУ, графический процессор, контроллеры периферии. Общая структура представлена на рисунке 3.1.

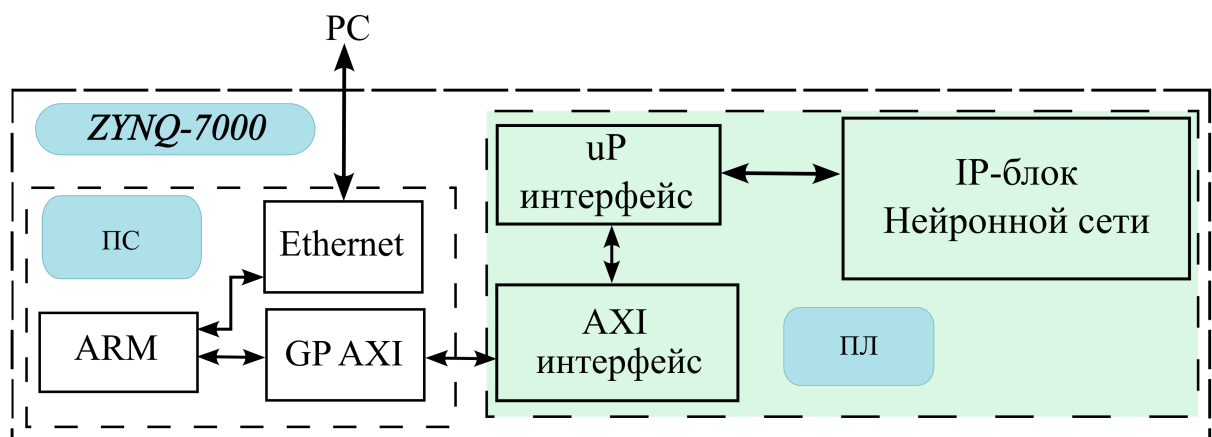


Рисунок 3.1 – Пример использования IP-блока нейронной сети

Последовательность обработки данных организуется следующим образом:

1 Исходное изображение размером 28×28 пикселей поступает на вход нейронной сети через переходник интерфейсов AXI-uP. Данные интерфейсы используются для взаимодействия с процессорной системой и обеспечивают передачу изображения в IP-блок. Пиксели изображения принимаются последовательно и записываются во внутреннюю память IP-блока, предназначенную для хранения изображения на протяжении всего цикла обработки.

2 Каждый отдельный пиксель изображения поочередно передаётся на вход массива из 28 умножающих-накопительных ядер (MAC), которые одновременно выполняют операции умножения входного значения пикселя на соответствующий весовой коэффициент. Весовые коэффициенты извлекаются из специализированных блоков памяти весов, которые хранят параметры, полученные в результате предварительного обучения модели.

3 За согласованность и последовательность операций отвечает управляющий модуль. Он обеспечивает адресацию ячеек памяти изображения и весов, организует подачу данных в вычислительные блоки и отслеживает фазы обработки. Управляющее устройство также координирует этапы переключения между слоями, а также формирует сигналы запуска и завершения вычислений.

4 После завершения расчетов в первом скрытом слое (LST-1), результат – новое представление входного изображения – передаётся во второй уровень вычислений. Здесь используются 10 MAC-ядер, каждое из которых обрабатывает соответствующий вектор признаков, поступающий от первого слоя. В результате получается выходной вектор размерности 10, где каждый элемент отражает степень принадлежности изображения к соответствующему классу (от 0 до 9).

5 Полученный выходной вектор подаётся в блок активации softmax. Этот модуль преобразует значения в вероятностную форму, нормируя их так, чтобы сумма элементов вектора была равна единице. На выходе формируется распределение вероятностей принадлежности изображения к каждому из 10 классов.

6 Индекс элемента вектора softmax, имеющий наибольшее значение (то есть наиболее вероятный класс), определяется и передаётся через интерфейс AXI4-lite обратно в процессорную систему. Этот интерфейс реализует простую и эффективную регистровую модель доступа, обеспечивая передачу управляющих команд и данных между IP-блоком нейронной сети и процессором. Интерфейс uP (user processor interface) позволяет интегрировать модель в более широкую систему на кристалле, облегчая её взаимодействие с внешним программным обеспечением и обеспечивая гибкость конфигурации [12].

Временная диаграмма работы uP-интерфейса представлена на рисунке 3.2.

AXI4-Lite – это упрощённый вариант AXI4-интерфейса, предназначенный для управления и обмена небольшими порциями данных. В отличие от AXI4, он поддерживает только однослотовые транзакции без разделения на несколько пакетов [11].

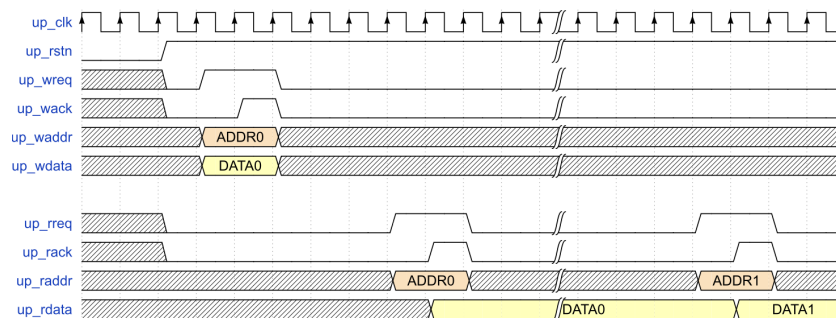


Рисунок 3.2 – Временная диаграмма транзакций чтения и записи интерфейса uP

Временная диаграмма работы AXI4-lite-интерфейса представлена на рисунке 3.3.

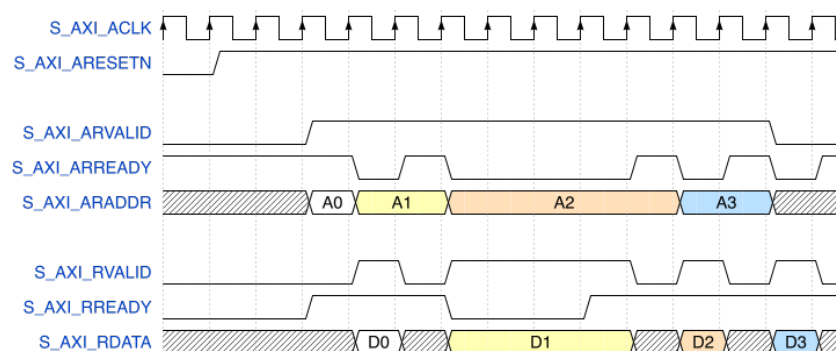


Рисунок 3.3 – Временная диаграмма транзакций чтения и записи интерфейса AXI4-lite

Оба интерфейса широко применяются в проектировании цифровых систем. uP-интерфейс удобен для работы с простыми IP-блоками, тогда как AXI4-Lite предоставляет стандартизированный метод взаимодействия с процессорными системами, такими как ARM-ядра на FPGA.

Электрическая структурная схема IP-блока нейронной сети прямого пространства приведена в графическом материале ГУИР 431289.002 Э1.

3.2 Программная реализация эталонной модели нейронной сети на языке Python

Программная реализация нейронной сети выполняется с использованием языка Python. В качестве эталонной модели реализуется нейронная сеть на основе библиотеки `fixpoint`, позволяющая провести сравнение точности вычислений при использовании чисел с фиксированной запятой и чисел с плавающей запятой.

Эталонная модель представляет собой нейросеть с двумя скрытыми слоями и активационной функцией `tanh`.

Для аппаратной реализации на FPGA и эталонной модели числа представляются в формате с фиксированной запятой (Fixed-Point). В отличие от чисел с плавающей запятой (Floating-Point), данный формат обеспечивает более эффективное использование аппаратных ресурсов, таких как блоки DSP и BRAM.

В данной работе используется фиксированное представление с разрядностью $Q_{6.7}$, где:

- 6 бит отведены под целую часть числа.
- 7 бит используются для дробной части.

Таким образом, числа в этом формате могут представлять значения в диапазоне $[-32, 31.996]$ с шагом $2^{-7} = 0.0078125$.

При переводе значений из плавающей запятой в фиксированную используется метод округления к ближайшему минимальному числу (округление вниз, или Round Down). Этот метод позволяет минимизировать ошибки округления и избежать систематического смещения, которое может возникать при округлении к ближайшему числу.

Такой метод округления позволяет повысить точность вычислений и уменьшить накопление ошибок при последовательных операциях, что критично для работы нейросети на FPGA.

Входные изображения размером 28×28 проходят последовательную обработку:

- Первый слой выполняет линейное преобразование входных данных с весами и смещением, после чего применяется функция активации тангенс.
- Второй слой аналогично выполняет линейное преобразование, используя параметры весов и смещений, после чего применяется функция активации тангенс.
- Выходной слой выполняет линейное преобразование входных данных с весами и смещением, а затем классифицирует изображение, вычисляя вероятности классов с помощью softmax.

Сравнение выходных значений между моделью с фиксированной точностью и моделью с плавающей точкой представлено на рисунке 3.4.

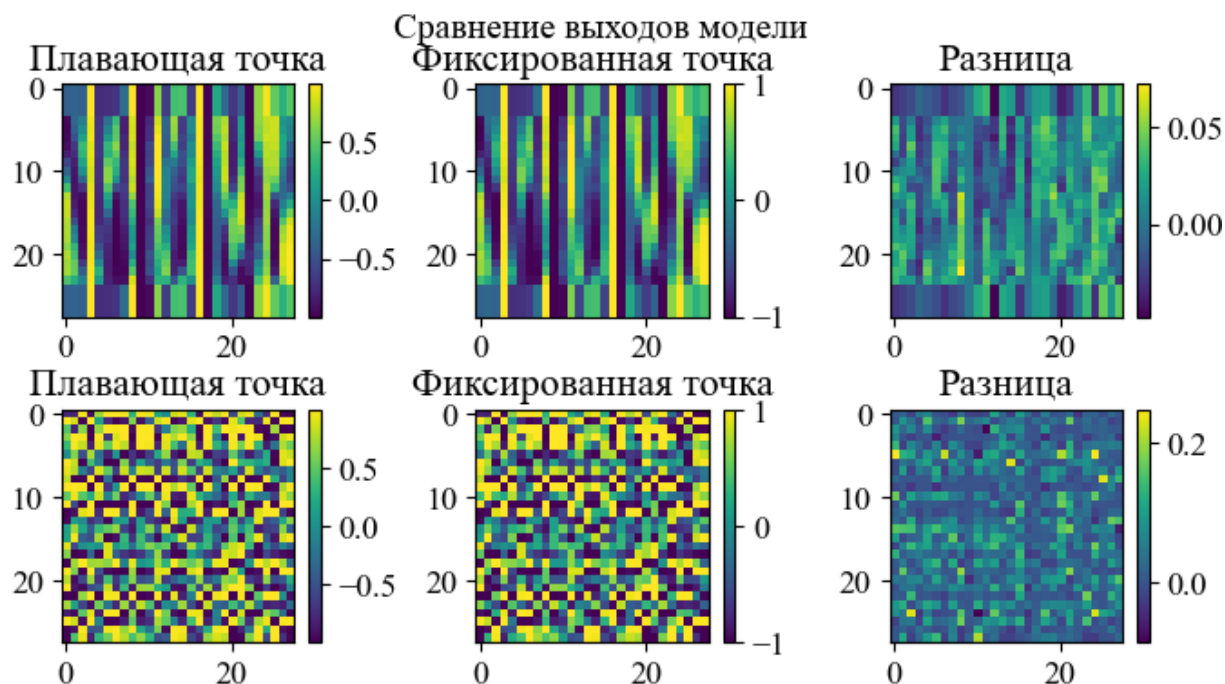


Рисунок 3.4 – Сравнение результатов вычислений

Графический анализ разности значений в обработке одного и того же изображения позволяет оценить влияние округлений и возможные потери точности.

Разработанная эталонная модель на Python позволяет проверить корректность вычислений перед аппаратной реализацией, а также провести тестирование точности чисел с фиксированной запятой.

Python описание нейронной сети с использованием библиотеки `fixpoint` приведено в приложении В.

3.3 Разработка операционной части нейронной сети

Нейронная сеть есть объединение управляющего и операционного автоматов, что показано на рисунке 3.5. Такое разбиение соответствует классическому подходу к построению цифровых вычислительных устройств, при котором логика работы системы делится на управление и выполнение.

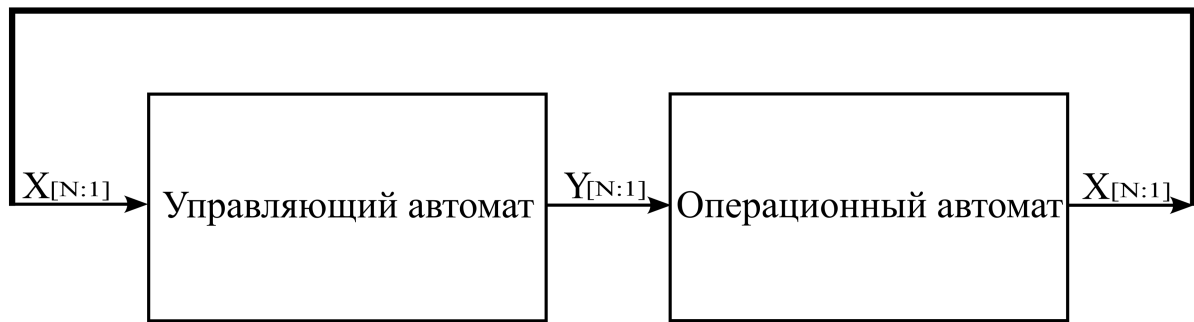


Рисунок 3.5 – Представление нейронной сети в виде управляющего и операционного автоматов

Элементарное действие, выполняемое в операционном автомате, называется микрооперацией. Пусть множество микроопераций, выполняемых под воздействием сигналов управляющего автомата, обозначается как $Y = y_1, \dots, y_N$.

Для выполнения микрооперации y_i ($i = 1, \dots, N$) необходимо появление соответствующего управляющего сигнала, принимающего значение единицы. Если в операционном автомате одновременно выполняется несколько микроопераций, они образуют микрокоманду.

Множество микрокоманд в совокупности с логическими условиями $X = x_1, \dots, x_N$, определяющими последовательность их выполнения, составляют микропрограмму. Логические условия соответствуют осведомительным сигналам, которые формируются в операционном автомате.

IP-блок нейронной сети должен быть спроектирован как независимый компонент системы, обеспечивающий модульность и возможность интеграции в различные вычислительные среды. Для этого необходимо, чтобы блок обладал четко определенным и простым интерфейсом, что позволит эффективно взаимодействовать с другими компонентами системы.

Интерфейс IP-блока должен обеспечивать передачу входных данных, управление процессом вычислений и выдачу результатов. Важно, чтобы структура взаимодействия соответствовала требованиям к гибкости и масштабируемости системы.

На рисунке 3.6 представлен требуемый вид интерфейса IP-блока, демонстрирующий основные входные и выходные сигналы.

Входные сигналы включают:

- clk – тактовый сигнал, синхронизирующий работу IP-блока;
- rst_n – активный по низкому уровню сигнал сброса, приводящий систему в исходное состояние;
- $start_i$ – управляющий сигнал запуска обработки входных данных;
- $addr_i$ – адресный сигнал, указывающий на текущую ячейку памяти изображения;
- $data_i$ – входные данные, представляющие собой значение пикселя изображения, поступающего в IP-блок.

Выходные сигналы включают:

- num_o – выходной сигнал, содержащий распознанное значение цифры (от 0 до 9), представляющее итог работы нейронной сети;
- rdy_o – сигнал готовности, свидетельствующий о завершении обработки и готовности результата к чтению со стороны процессорной системы.



Рисунок 3.6 – Интерфейс IP-блока нейронной сети

3.4 Проектирование функциональной схемы нейронной сети

При разработке IP-блока нейронной сети необходимо определить его функциональную структуру, обеспечивающую корректное выполнение вычислений, взаимодействие с внешними компонентами системы и эффективное управление процессом обработки данных.

Функциональная схема IP-блока должна описывать основные модули и их взаимодействие, включая блоки обработки входных данных, вычислительное ядро нейронной сети, управляющий модуль и интерфейсы для связи с внешней средой. Важно учитывать требования к производительности, ресурсоемкости и масштабируемости, чтобы обеспечить возможность интеграции IP-блока в различные аппаратные платформы.

Данная нейронная сеть состоит из нескольких основных блоков. Существует два типа блоков обработки пикселя, которые отличаются количеством блоков памяти, которые должны обрабатываться мультиплексором и поступать в качестве входных данных на умножитель.

После вычисления полносвязного слоя для конкретной строки или столбца данные поступают в регистр обработки, состоящий из 28 чисел разрядностью 13 бит.

Затем данные из этого регистра последовательно поступают в регистр данных тангенса, из которого они отправляются в блок аппроксимации тангенса гиперболического.

Также используются два пяти разрядных и один десяти разрядный счетчики, которые используются для генерации адреса в память весов и в память изображения.

Блок функции активации softmax реализован на принципе сравнения всех

десяти входных классов и выбора наибольшего значения, что будет соответствовать наиболее вероятному значению. В результате на выход поступает индекс максимального элемента.

Для управления работой и генерирования микроопераций на основе логических условий используется микропрограммный автомат.

Электрическая функциональная схема IP-блока нейронной сети прямого распространения приведена в графическом материале ГУИР 431289.003 Э1.

Электрическая функциональная схема блоков обработки пикселей приведена в графическом материале ГУИР 431289.004 Э1.

3.5 Построение алгоритма работы нейронной сети

На данном этапе проектирования создается алгоритм, определяющая последовательность выполнения операций внутри IP-блока нейронной сети. Она описывает порядок обработки входных данных, передачу информации между модулями и выполнение вычислений.

Алгоритм строится на основе дискретных состояний, в которых выполняются конкретные микрооперации. Каждое состояние характеризуется активными управляющими сигналами, обеспечивающими выполнение определенной части вычислений.

Основные этапы работы алгоритма включают:

1 Загрузку изображения в память – входные данные загружаются в буфер оперативной памяти, откуда они будут переданы в вычислительные блоки.

2 Выполнение операций LST-преобразования – производится начальная обработка изображения, включающая нормализацию и предварительное выделение признаков.

3 Передача данных в полносвязный слой – результаты преобразования передаются в полносвязный слой нейросети для дальнейшей обработки.

4 Выполнение финальной классификации – производится анализ данных нейросетью и определение класса входного изображения.

5 Отправка данных в систему обработки – результаты классификации передаются в систему для последующего использования.

Алгоритм работы нейронной сети по распознаванию рукописных цифр представлен в графическом материале ГУИР 431289.005 ПД.

3.6 Проектирование управляющей части

Проектирование управляющей части вычислительного устройства выполняется с применением метода структурирования конечных состояний, который широко используется при разработке цифровых систем управления.

Используется канонический метод структурного синтеза микропрограммного автомата (МПА), основанный на синхронной логике. Данный метод обеспечивает формальное построение управляющей логики устройства на основе

конечного множества состояний, переходов между ними и соответствующих управляющих сигналов.

МПА делится на два основных структурных компонента:

1 Память управляющих микрокоманд, в которой хранится заданная последовательность управляющих слов. Каждое управляющее слово определяет активность тех или иных сигналов на каждом такте работы устройства, в зависимости от текущего состояния.

2 Комбинационная схема управления, которая отвечает за генерацию адреса следующей микрокоманды. Адрес определяется на основе текущего состояния автомата и внешних условий. Это позволяет гибко управлять переходами между состояниями в зависимости от динамики процесса обработки.

На рисунке 3.7 представлена обобщённая структурная схема такого микропрограммного автомата.

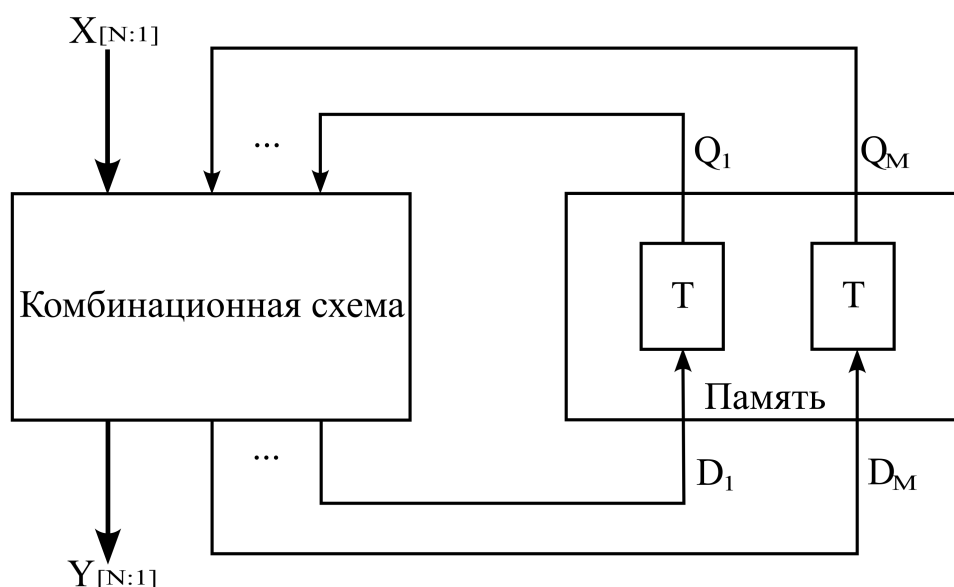


Рисунок 3.7 – Представление управляющей части нейронной сети в виде композиции памяти и комбинационной схемы

Память автомата состоит из заранее выбранных элементов памяти, обеспечивающих хранение информации о текущем состоянии устройства. В качестве элементов памяти используются триггеры, которые позволяют надежно фиксировать и изменять состояние системы.

Количество элементов памяти, необходимых для хранения состояний автомата, определяется по следующей формуле:

$$N = \log_2 M, \quad (3.1)$$

где M – число состояний микропрограммного автомата.

Однако в практических реализациях часто применяется кодирование

состояний с использованием кода Грея. Код Грея представляет собой двоичную систему кодирования, в которой любые два последовательных числа отличаются изменением только одного бита[13]. Это позволяет уменьшить количество переключений триггеров при переходе между состояниями, снижая вероятность ошибок, возникающих из-за временных несовпадений сигналов.

Дополнительно, в разработке автоматов управления применяется стратегия, при которой старший бит кодового слова указывает на нахождение автомата в состоянии ожидания. Это состояние используется для инициализации системы, сброса счетчиков и минимизации нежелательных переходов. Применение старшего бита в качестве указателя на режим ожидания помогает уменьшить влияние гличей (glitch) на граф переходов. Гличи – это кратковременные нежелательные изменения сигнала, возникающие при переключении состояний из-за различий во времени распространения сигналов через логические элементы [14]. Они могут приводить к некорректным переходам в автомате, вызывая ошибки в работе системы.

В связи с этим, традиционная формула определения числа элементов памяти модифицируется следующим образом:

$$N = \log_2 M + 2 = \log_2 16 + 2 = 6, \quad (3.2)$$

где $M = 16$ – число состояний микропрограммного автомата.

Каждое состояние автомата описывается представлением из шести бит, что позволяет повысить надежность работы системы, уменьшить количество переключений триггеров и снизить вероятность ошибок, связанных с гличами.

3.7 Построение графа переходов управляющего автомата

Граф переходов управляющего автомата позволяет представить алгоритм в соответствии с условными обозначениями, что представлено на рисунке 3.8.

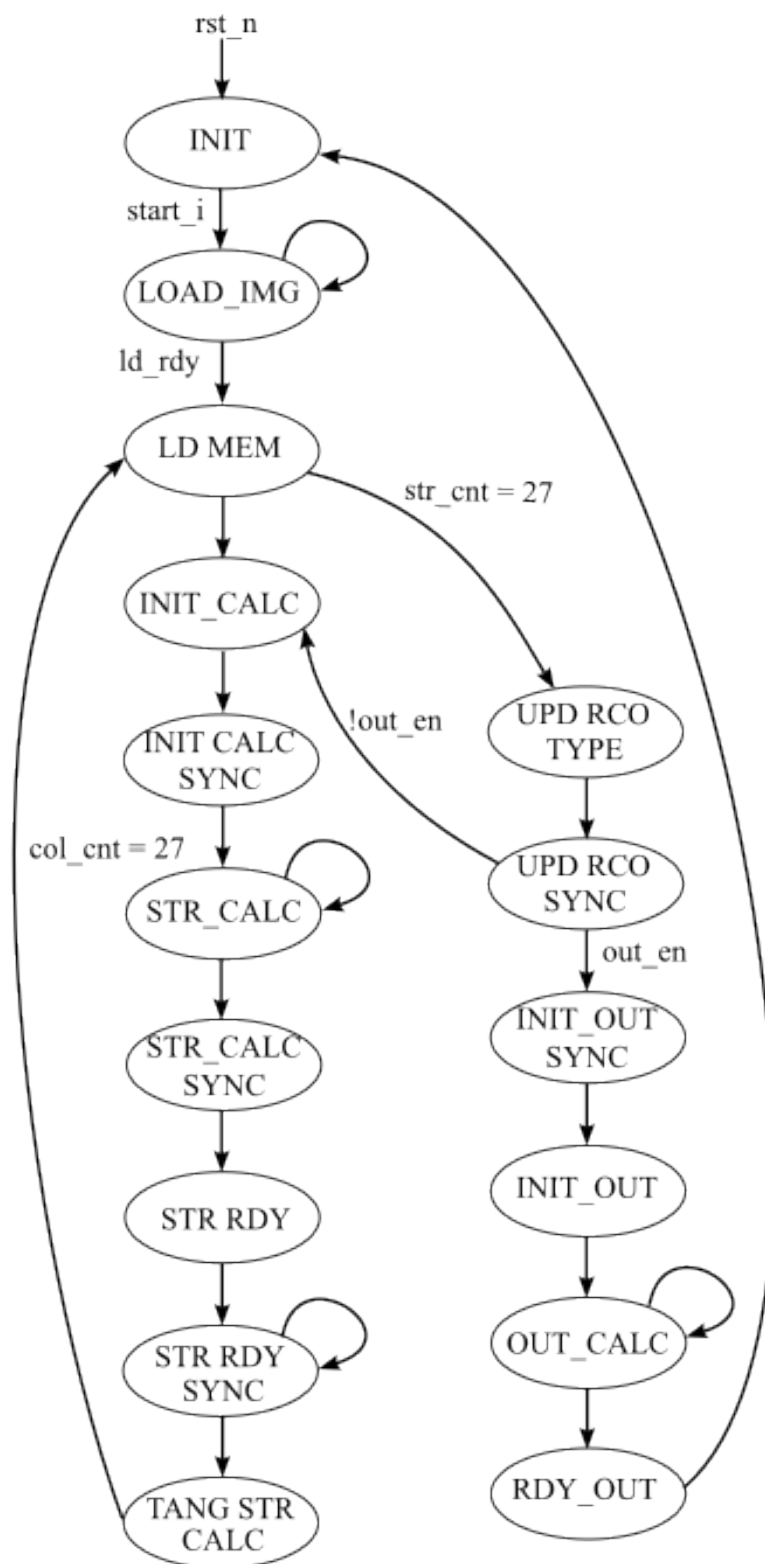


Рисунок 3.8 – Граф переходов

Такая форма представления облегчает анализ работы системы и позволяет четко проследить взаимосвязь между различными этапами обработки данных. Для каждого состояния определено mnemonic обозначение, которое обеспечивает удобство представления и понимания процесса работы автомата. Все состояния кодируются в виде шести битных значений, сформированных в коде

Грея.

Состояния граф-схемы алгоритма объединены в блоки:

1 INIT – инициализация системы, установка начальных параметров, необходимых для корректной работы всех последующих этапов.

2 LOAD – загрузка изображения.

3 LD_MEM – подготовка данных.

4 INIT_CALC_SYNC – синхронизация смещений.

5 INIT_CALC – инициализация смещения.

6 STRING_CALC – обработка строки/столбца.

7 STRING_SYNC – синхронизация обработка последнего пикселя.

8 STRING_RDY – готовность результата обработки.

9 STRING_RDY_SYNC – синхронизация флага готовности.

10 TG_STRING_CALC – вычисление тангенса.

11 UPD_RCO_TYPE – смена слоя.

12 UPD_RCO_SYNC – синхронизация смены слоя.

13 INIT_OUT_SYNC – синхронизация выходных смещений.

14 INIT_OUT – инициализация выходных смещений.

15 OUT_CALC – обработка выходного полносвязного слоя.

16 OUT_RDY – готовность детектирования.

4 АППАРАТНАЯ РЕАЛИЗАЦИЯ IP-БЛОКА НЕЙРОННОЙ СЕТИ ДЛЯ РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР

4.1 Описание пакета Xilinx Vivado

Xilinx Vivado – это современный программный пакет для проектирования цифровых схем на базе программируемых логических интегральных схем, разработанный компанией Xilinx. Данный программный комплекс является преемником Xilinx ISE и значительно превосходит его по возможностям и производительности.

Vivado предназначен для автоматизированного проектирования цифровых схем, начиная с этапа описания логики на языках описания аппаратуры и заканчивая загрузкой сгенерированного битового потока в устройство[15].

Одним из ключевых преимуществ Vivado является использование более производительных алгоритмов синтеза и оптимизации, что позволяет существенно ускорить процесс разработки. В отличие от традиционных средств проектирования, в Vivado реализована поддержка параллельных вычислений, что делает его особенно эффективным при работе с большими проектами.

В состав пакета Vivado входят несколько инструментов, среди которых[15]:

1 Vivado IDE – интегрированная среда разработки, предоставляющая удобный графический интерфейс для управления проектами, анализа схемотехнической логики и моделирования.

2 Vivado Synthesis – инструмент синтеза логики, который преобразует код HDL в оптимизированное представление на уровне вентилей.

3 Vivado Implementation – модуль размещения, трассировки и оптимизации логики на кристалле FPGA.

4 Vivado Simulator – встроенный инструмент для моделирования цифровых схем на уровне поведенческой, функциональной и временной верификации.

5 Vivado IP Integrator – инструмент для работы с IP-ядрами и объединения их в сложные системы на кристалле.

6 Vivado High-Level Synthesis (HLS) – средство высокоуровневого синтеза, позволяющее описывать аппаратные алгоритмы на языках C/C++ и автоматически преобразовывать их в оптимизированный RTL-код.

Основные возможности Vivado:

1 Поддержка языков описания аппаратуры VHDL и Verilog. Vivado позволяет разрабатывать схемы на двух основных языках HDL, а также поддерживает SystemVerilog для верификации цифровых проектов. Это дает широкие возможности для разработки и тестирования аппаратуры.

2 Автоматизированный синтез и размещение логики на FPGA. Vivado включает мощный механизм синтеза, который автоматически преобразует HDL-код в оптимизированную сетевую схему, выполняет ее размещение на кристалле FPGA и прокладывает соединения между логическими блоками.

3 Встроенные инструменты для моделирования и отладки.

4 Генерация IP-ядер и использование встроенных библиотек компонентов.

Для реализации нейронной сети Vivado используется на нескольких ключевых этапах разработки. Во-первых, он применяется для генерации аппаратной логики, необходимой для выполнения расчетов внутри FPGA. Во-вторых, с его помощью производится компиляция проекта и синтез аппаратной схемы. В-третьих, Vivado используется для загрузки сгенерированного битового потока в ПЛИС и отладки работы модели.

4.2 Описание функциональных блоков

Аппаратная реализация нейронной сети включает несколько ключевых функциональных блоков:

- 1 Блок обработки пикселя.
- 2 Блок вычисления тангенса.
- 3 Блок вычисления softmax.
- 4 Блок памяти изображения.
- 5 Блок управляющего автомата.
- 6 Блок MAC.

Каждый из этих блоков реализуется в виде аппаратного модуля на HDL.

В блоке обработки пикселя осуществляется установка параметра смещения для подготовки к вычислению слоя, а также выбирается соответствующий данному этапу весовой коэффициент из памяти.

В блоке вычисления тангенса осуществляется аппроксимированный расчет функции активации тангенс гиперболический для LST-1 слоя.

Прямая реализация вычисления тангенса будет занимать большое количество вычислительных ресурсов. Поэтому для удобства реализации используется аппроксимация в соответствии с формулой[16]:

$$\tanh(x) \approx F(x) = \begin{cases} \text{sign}(x), & \text{если } |x| > 2, \\ (1 + \frac{x}{4}) \cdot x, & \text{если } -2 < x < 0, \\ (1 - \frac{x}{4}) \cdot x, & \text{если } 0 < x < 2. \end{cases} \quad (4.1)$$

В блоке вычисления softmax происходит сравнение входных данных и определение наиболее вероятного класса.

В блоке памяти изображения храниться результат обработки на каждом этапе. От приемки входных данных, до выхода LST-1 слоя.

Блок управляющего автомата осуществляет управление и синхронизацию между всеми блоками устройства и обрабатывает входные управляющие сигналы.

Блок MAC строится на основе блока DSP48. Данный блок используется для вычисления операции умножения с накоплением. В данном блоке по сигналу инициализации устанавливается смещение, необходимо для текущего этапа

вычислений, а затем выполнение операции MAC на каждом такте синхронизации. Реализация данных блоков на языке SystemVerilog представлена в приложении Г.

4.3 Описание интерфейса нейронной сети

Как было показано ранее на рисунке 4.5 нейронная сеть состоит из пяти обязательных входных и двух выходных сигналов. Взаимодействие процессорной системы и программируемой логики осуществляется через интерфейс AXI4-lite.

Необходимым условием, выдвигаемым к IP-блоку будет поддержка интерфейса AXI4-lite. Для упрощения формирования IP-блока будем использовать связку AXI-uP. uP интерфейс напрямую взаимодействует с модулем верхнего уровня. Далее данные передаются в AXI-slave модуль, который и подключается к процессорной системе.

4.4 Описание процесса создания блочной диаграммы

Рассмотрим основные задачи связанные с созданием блочной диаграммы (Block diagram), подключением готовых IP-ядер и созданием собственных, а также созданием топ-модуля на языке Verilog. Создадим блочную диаграмму, что показано на рисунке 4.1.

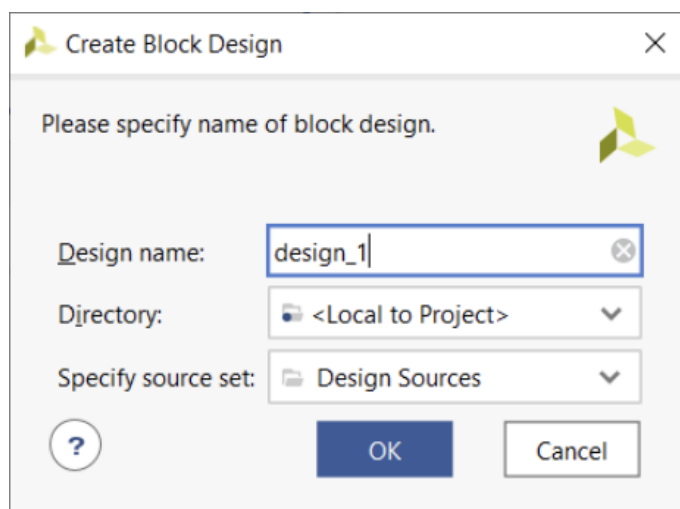


Рисунок 4.1 – Создание блочной диаграммы

Перейдем к созданию собственного IP-ядра, что показано на рисунке 4.2

Create and Package New IP

Peripheral Details

Specify name, version and description for the new peripheral

Name:

m_nn

Version:

1.0

Display name:

m_nn_v1.0

Description:

NN IP block

IP location:

/home/ek/Documents/nn/ip_repo

☐ Overwrite existing

?

< Back

Next >

Finish

Cancel

Рисунок 4.2 – Создание IP-ядра в САПР Vivado

Далее выполним добавление всех файлов в IP-блок, что представлено на рисунке 4.3

Packaging Steps

Identification

Compatibility

File Groups

Customization Parameters

Ports and Interfaces

Addressing and Memory

Customization GUI

Review and Package

File Groups

Name	Library Name	Type	Is Include	File Group Name	Model Name
Standard			☐		
Synthesis (13)			☐		m_nn_v1_0
src/up_axi.v		verilog	☐	xilinx_anylanguagesynthesis	
src/PE_rc.sv		syster	☐	xilinx_anylanguagesynthesis	
src/PE_rco.sv		syster	☐	xilinx_anylanguagesynthesis	
src/ROM_col.sv		syster	☐	xilinx_anylanguagesynthesis	
src/ROM_row.sv		syster	☐	xilinx_anylanguagesynthesis	
src/ROM_w_out.sv		syster	☐	xilinx_anylanguagesynthesis	
src/nn_param_pkg		syster	☐	xilinx_anylanguagesynthesis	
src/fc.sv		syster	☐	xilinx_anylanguagesynthesis	
src/img_mem.sv		syster	☐	xilinx_anylanguagesynthesis	
src/mac_core.sv		syster	☐	xilinx_anylanguagesynthesis	
src/softmax.sv		syster	☐	xilinx_anylanguagesynthesis	
src/tanh_function.		syster	☐	xilinx_anylanguagesynthesis	
hdl/m_nn_v1_0.v		verilog	☐	xilinx_anylanguagesynthesis	
Simulation (13)			☐		m_nn_v1_0
Advanced			☐		
Software Driver (6)			☐		
UI Layout (1)			☐		
Block Diagram (1)			☐		

Рисунок 4.3 – Добавление файлов

Далее необходимо перейти к построению блочной диаграммы аппаратной части проекта в среде Vivado. Блочная диаграмма представляет собой графиче-

ское описание аппаратных соединений между IP-ядрами, процессорной системой и периферией.

Для обеспечения связи между несколькими IP-блоками и процессором в систему добавляется модуль AXI Interconnect. Этот модуль играет ключевую роль в маршрутизации данных, поступающих от мастер-устройств к одному или нескольким ведомым (slave) IP-блокам. Он автоматически адаптирует адресацию, сигналы управления и синхронизации, обеспечивая корректный обмен данными в системе.

Также обязательным элементом системы является Processor System Reset. Этот блок управляет сигналами сброса всех модулей, входящих в состав блочной диаграммы. Он обеспечивает корректную и синхронную инициализацию всех компонентов после подачи питания или программного сброса, предотвращая возможные ошибки при старте системы.

На рисунке 4.4 показан результирующий вид блочной диаграммы, содержащей процессорную систему, пользовательский IP-блок, AXI Interconnect и блок сброса. Все соединения между модулями реализованы с соблюдением требований интерфейсов AXI и рекомендаций Vivado.

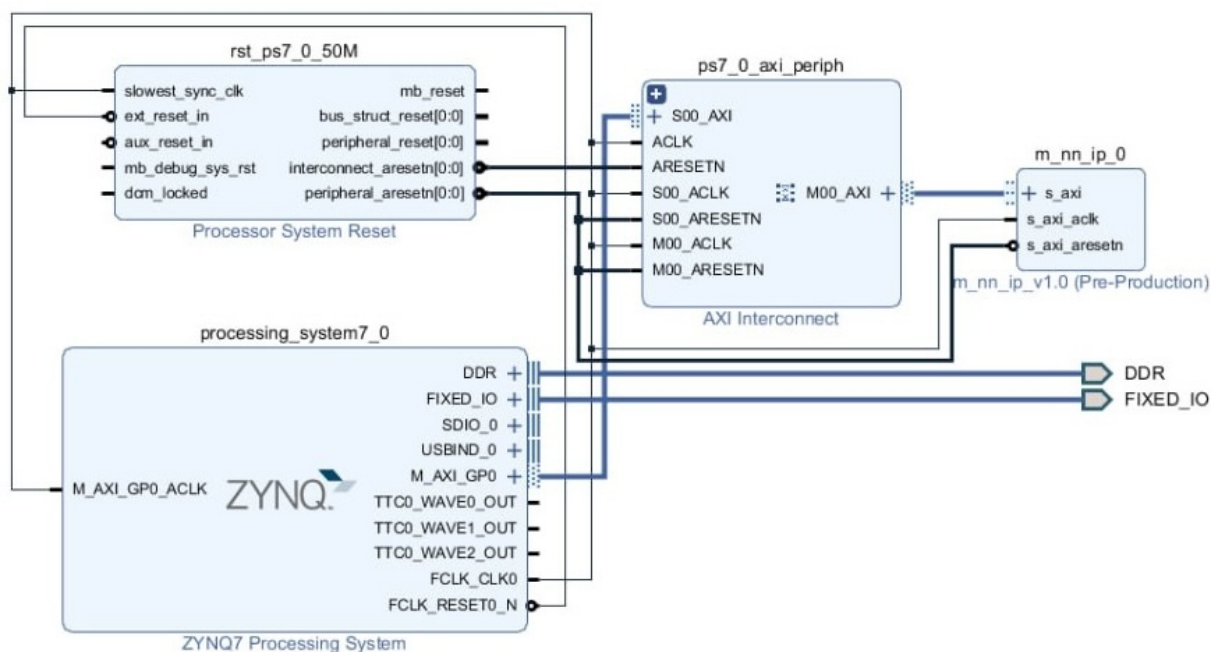


Рисунок 4.4 – Результирующий вид блочной диаграммы

Для корректной работы системы требуется обеспечить стабильный тактовый сигнал с частотой 80 МГц. По умолчанию, тактовый генератор в Zynq-процессорной системе может выдавать несколько частот, но для нужд пользовательского IP-блока необходима именно частота 80 МГц.

Для этого на вкладке Clock Configuration в настройках Processing System 7 выполняется установка пользовательской частоты. Один из выходных тактовых сигналов (FCLK_CLK0) настраивается на значение 80 МГц. Этот сигнал затем

подаются на все соответствующие модули системы, включая пользовательский IP-блок, обеспечивая их синхронную работу.

Процесс настройки частоты подробно продемонстрирован на рисунке 4.5.

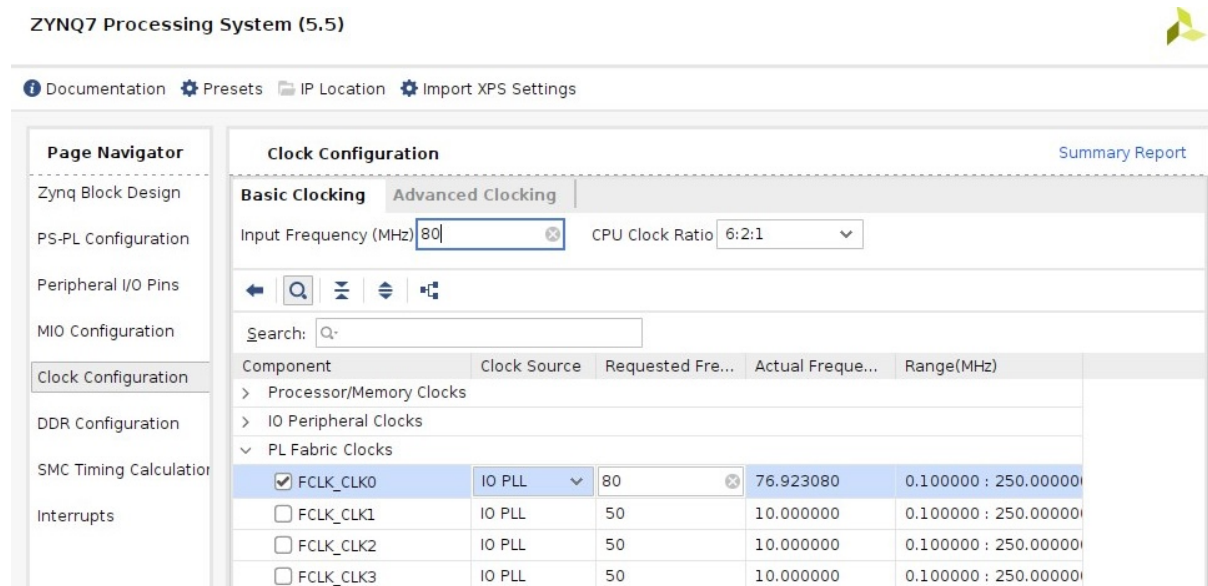


Рисунок 4.5 – Установка частоты 80 МГц в блоке Processing System

4.5 Результаты синтеза и симуляции

Для проверки работоспособности написанного устройства был составлен тест, в котором формируются управляющие воздействия и изображение. Работа устройства начинается по сигналу start_i.

Для тестирования было взято два изображения, что показано на рисунке 4.6.



Рисунок 4.6 – Тестируемые изображения

Тест представлен в приложении Г. Для проверки корректности работы нейронной сети необходимо сравнить выход результирующего полносвязного слоя и нейронной сети.

На рисунках 4.7 и 4.8 представлены временные диаграммы поведенческого моделирования описанной на SystemVerilog нейронной сети и выход эталонной нейронной сети на языке Python.

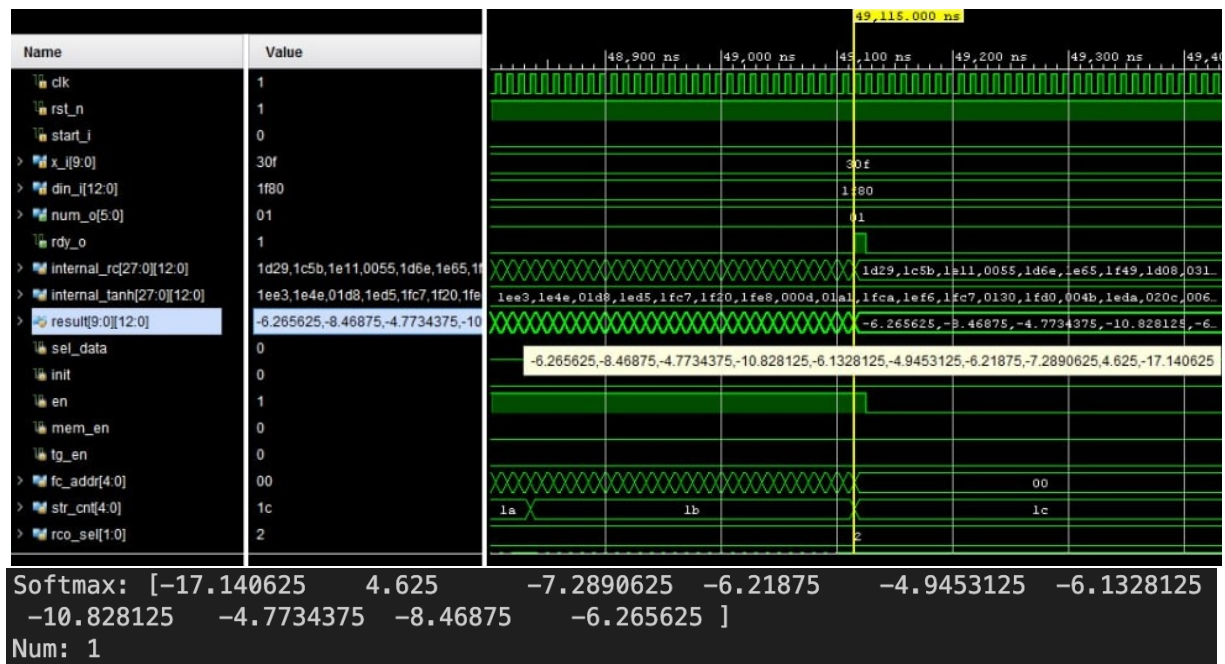


Рисунок 4.7 – Временная диаграмма нейронной сети и результат работы эталона на первом изображении

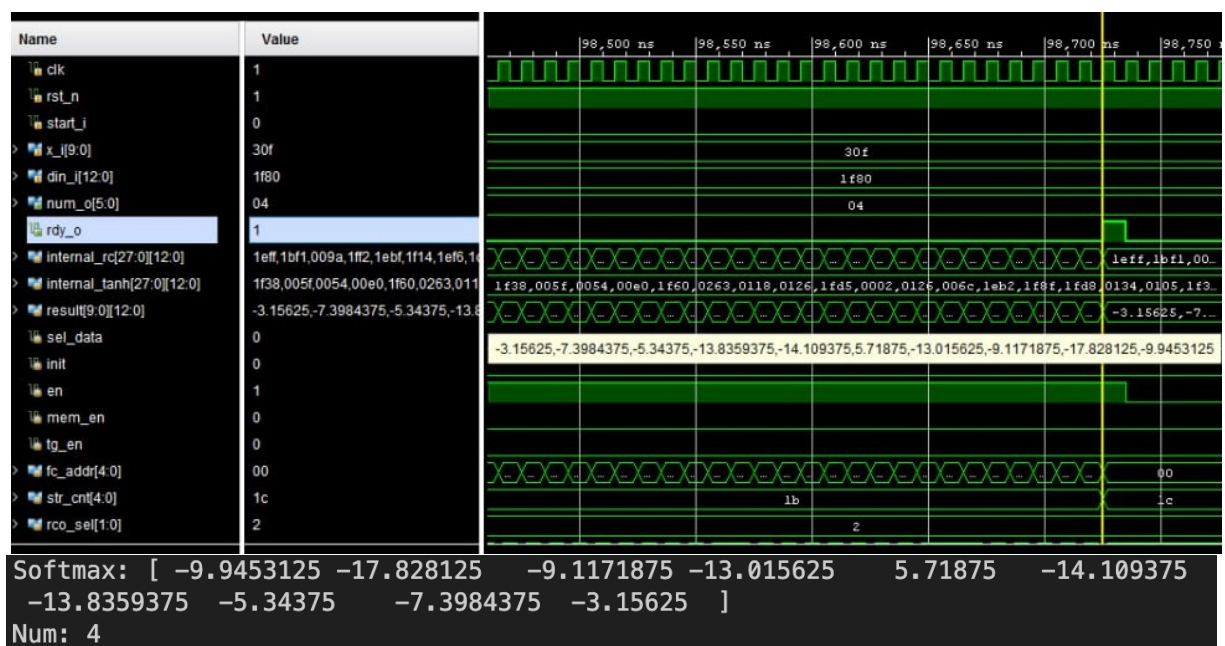


Рисунок 4.8 – Временная диаграмма нейронной сети и результат работы эталона на втором изображении

В результате проведенного моделирования и сопоставления работы реализованной аппаратной модели с эталонной моделью на языке Python с использованием библиотеки fixpoint было выявлено полное соответствие выходных данных. Это подтверждает корректность проектирования и реализации нейронной сети

на уровне RTL-описания и её работоспособность при выполнении на целевой ПЛИС.

Для оценки ресурсоёмкости и эффективности реализации проекта был проведён анализ использования аппаратных ресурсов, таких как логические элементы (LUT), триггеры (FF), специализированные арифметические блоки (DSP) и память (BRAM), полученные в процессе синтеза HDL-описания.

Полученные результаты представлены на рисунке 4.9.

Resource	Utilization	Available	Utilization %
LUT	1302	17600	7.40
LUTRAM	60	6000	1.00
FF	1461	35200	4.15
BRAM	33.50	60	55.83
DSP	57	80	71.25
BUFG	1	32	3.13

Рисунок 4.9 – Аппаратные затраты

Для дополнительной верификации корректности функционирования разработанной системы проводится временная симуляция на основе результатов синтеза и имплементации.

На данном этапе осуществляется моделирование с учетом временных задержек, рассчитанных инструментом синтеза, однако ещё без учёта конкретных физических ресурсов устройства.

Постсинтезная симуляция позволяет убедиться, что логическая структура проекта сохранена и корректно функционирует после преобразования исходного RTL-кода в структуру из примитивов целевой ПЛИС.

Для сохранения структуры блоков при синтезе используются атрибуты, которые запрещает среде Xilinx Vivado осуществлять смешивание функциональных блоков с целью оптимизации:

```
(* dont_touch = "yes" *) (* keep_hierarchy = "yes" *)
```

Результаты такой симуляции представлены на рисунке 4.10. Видно, что формируемые сигналы соответствуют ожидаемым значениям, соблюдаются временные зависимости и интерфейсы взаимодействуют корректно.

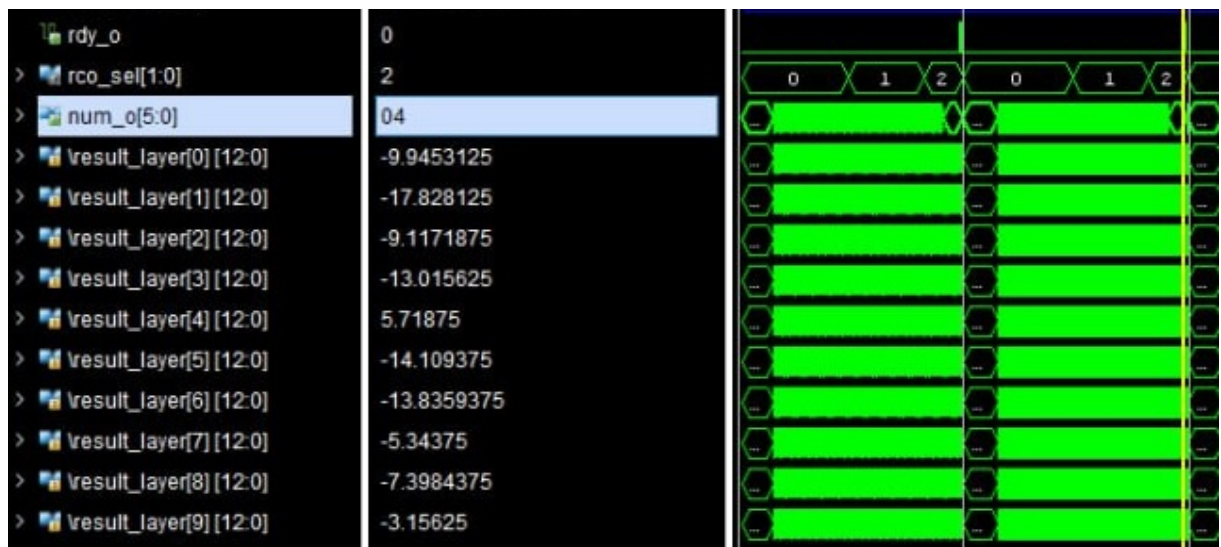


Рисунок 4.10 – Временная диаграмма нейронной сети на втором изображении при пост-синтез симмуляции

После завершения этапа имплементации, включающего размещение и трассировку (place & route), производится повторная временная симмуляция, уже с учетом всех физических задержек, связанных с реальной топологией на кристалле. Это позволяет оценить окончательные временные характеристики и убедиться, что проект не нарушает временных ограничений (timing constraints).

На рисунке 4.11 показана временная диаграмма, соответствующая данному этапу. Анализ сигналов подтверждает корректную работу всех блоков системы, несмотря на добавление реальных задержек межсоединений и элементов логики.

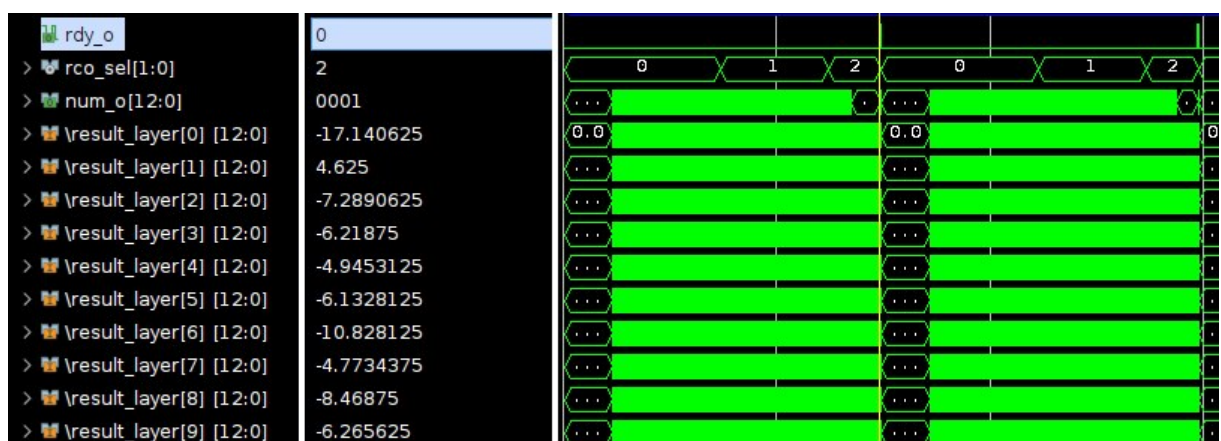


Рисунок 4.11 – Временная диаграмма нейронной сети на первом изображении при пост-имплементация симмуляции

Ещё одним важным параметром при анализе аппаратной реализации является потребляемая мощность, которая играет ключевую роль при выборе целевого устройства и оценке общей энергоэффективности системы.

На рисунке 4.12 представлена оценка энергопотребления реализованной нейронной сети на основе результатов имплементации.

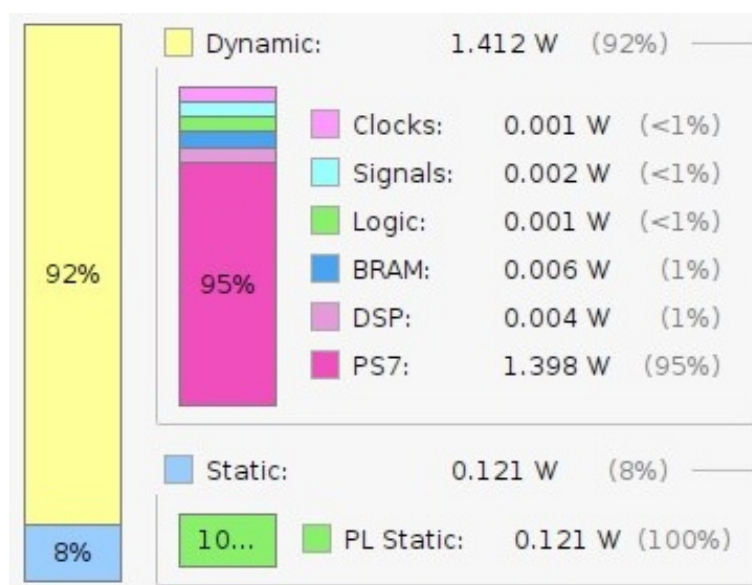


Рисунок 4.12 – Оценка потребляемой мощности

Общая мощность составляет 0.166 Вт, из которых:

1 Статическая мощность (Static) — 0.095 Вт, что составляет 57% от общего потребления. Это базовая мощность, потребляемая устройством при включении, вне зависимости от его активности.

2 Динамическая мощность (Dynamic) — 0.071 Вт (43%), обусловлена непосредственной работой схем, переключением сигналов, выполнением операций и передачей данных.

Результаты проектирования сгруппированы и представлены в графическом материале ГУИР 431289.006 ПЛ.

5 АНАЛИЗ РЕЗУЛЬТАТОВ ТЕСТИРОВАНИЯ IP-БЛОКА НЕЙРОННОЙ СЕТИ ДЛЯ РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР

5.1 Описание среды тестирования

В качестве аппаратной платформы для реализации и тестирования разработанных решений использовалась плата на базе программируемой логической интегральной схемы ZYBO Z7 от компании Digilent. Плата ZYBO Z7 представляет собой одноплатный компьютер, основанный на СнК Xilinx Zynq-7000, которая сочетает в себе двухъядерный процессор ARM Cortex-A9 и среду программируемой логики (PL) на одном чипе[9].

ПЛИС ZYBO Z7 оснащена широким набором периферийных интерфейсов, таких как HDMI, USB, Ethernet, аудиоразъёмы и слот для карты памяти microSD. Такая конфигурация позволяет использовать плату как для высокопроизводительных вычислений, так и для задач обработки сигналов и изображений в реальном времени. Программируемая часть ПЛИС используется для создания специализированных аппаратных ускорителей обработки данных, а процессорная часть (Processing System, PS) обеспечивает управление и взаимодействие между различными компонентами системы[17].

Для упрощения разработки приложений и взаимодействия между процессорной системой и пользовательскими IP-блоками программируемой логики, в работе применялась среда PYNQ (Python Productivity for Zynq). Общая структура использования PYNQ для разработки Zynq представлена на рисунке 5.1 [18].

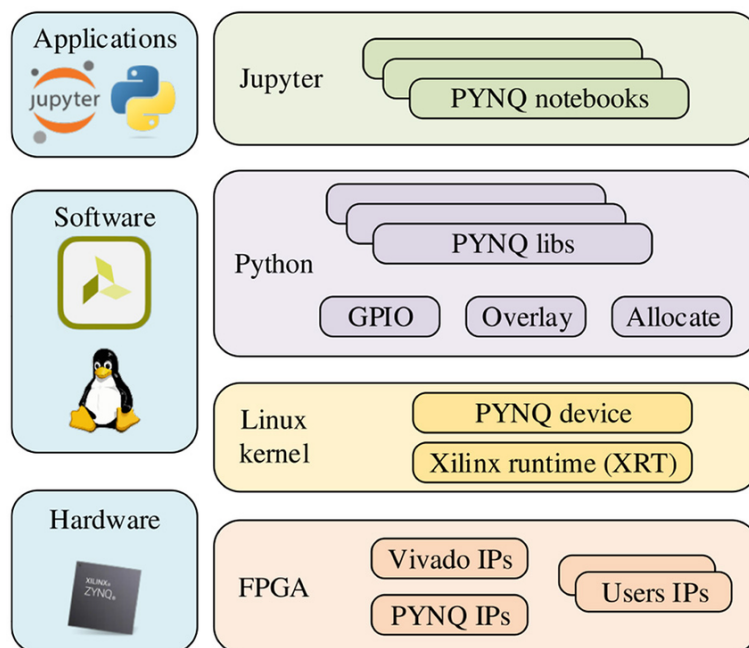


Рисунок 5.1 – Зависимость точности от разрядности

PYNQ представляет собой специализированную программную платформу, основанную на операционной системе Linux, которая позволяет использовать язык программирования Python для управления аппаратными ресурсами Zynq[19]. PYNQ предоставляет удобный интерфейс для работы с IP-ядрами, созданными в среде Vivado, через использование библиотек и драйверов высокого уровня.

Программная часть проекта выполнялась на процессоре ARM, работающем под управлением операционной системы PYNQ Linux. Управление аппаратными IP-блоками осуществлялось через Python API, что позволило оперативно тестировать и модифицировать аппаратные модули без необходимости перекомпиляции всего проекта. Благодаря этому подходу существенно ускорился процесс отладки и тестирования системы.

5.2 Тестирование разработанного IP-блока нейронной сети

Тестирование осуществлялось посредством запуска специально разработанного скрипта на языке Python в среде PYNQ.

Основной задачей тестирования являлась проверка корректности выполнения операций нейронной сети, реализованных в аппаратной логике, и взаимодействия IP-блока с процессорной системой через интерфейсы AXI4-lite и uP. Для этого использовался скрипт на Python, который выполнял следующие функции:

- 1 Инициализация IP-блока нейронной сети.
- 2 Загрузка тестовых данных во входные регистры IP-блока.
- 3 Запуск аппаратной обработки данных.
- 4 Считывание результатов вычислений из выходных регистров.

Также отдельно осуществляется сравнение полученных результатов с эталонными значениями для верификации корректности работы.

На этапе инициализации скрипт с помощью предоставленных библиотек PYNQ, таких как Overlay, загружает конфигурацию программируемой логики и получал доступ к IP-блоку.

Для подачи входных данных использовался архив с тестовыми изображениями из базы MNIST. Осуществляется чтение изображения и значения. Затем изображение нормализуется в соответствии с описанными ранее требованиями. Скрипт производит запись данных во входные регистры через AXI-интерфейс и инициировал начало вычислений путём установки управляющих флагов.

По завершении обработки скрипт ожидает сигнал готовности от IP-блока. После этого результаты обработки считывались из выходных регистров. Полученные данные сравнивались со считанным значением из базы.

В тестовом окружении необходимо подключить библиотеку PYNQ и файл с прошивкой:

```
from pynq import Overlay
```

```
platform = Overlay("design_1.bit")
neural_net = platform.m_nn_0
```

Полный скрипт по обработке изображения на ПЛИС представлен в приложении Д.

На рисунке 5.2 представлен пример работы данного скрипта.

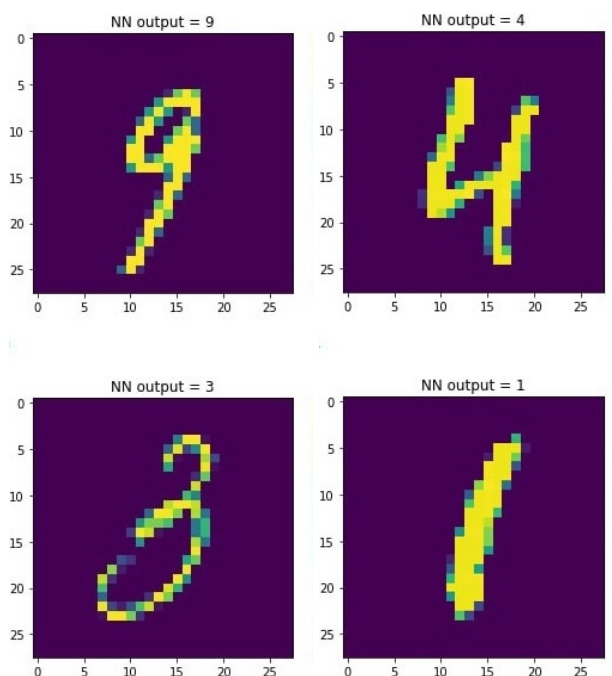


Рисунок 5.2 – Результаты распознавания рукописных цифр

5.3 Экспериментальное исследование IP-блока нейронной сети

Для оценки эффективности работы разработанного IP-блока нейронной сети было проведено три последовательных эксперимента. Эксперименты направлены на проверку корректности работы системы, оценку точности классификации и исследование влияния разрядности весовых коэффициентов на качество работы сети. Произведено три эксперимента:

1 Тестирование на 10000 изображений из MNIST. Построение матрицы спутывания и сравнение с эталоном.

2 Использование консольного приложения по рисованию на холсте с возможностью сохранить изображение в формате 28 на 28 и дальнейшей обработке на ПЛИС.

3 Изменение разрядности представления весовых коэффициентов и проверка зависимости точности от разрядности.

Скрипт представлен в приложении Д.

5.3.1 Первым этапом тестирования стала проверка IP-блока на большом количестве стандартных данных. Для этого использовалась тестовая выборка из

10000 изображений базы данных MNIST, содержащей изображения рукописных цифр размером 28×28 пикселей.

Каждое изображение подавалось на вход разработанному IP-блоку, после чего считывался результат классификации. На основании полученных ответов была построена матрица спутывания, отражающая количество верных и ошибочных классификаций для каждого класса.

Матрица спутывания позволяет оценить, какие именно классы наиболее часто путаются между собой, а также вычислить стандартные метрики качества классификации, такие как точность, полнота и специфичность.

Сравнение результатов работы IP-блока с эталонной программной моделью нейронной сети показало полное соответствие, что свидетельствует о правильности аппаратной реализации.

На рисунке 5.3 представлены матрицы спутывания.

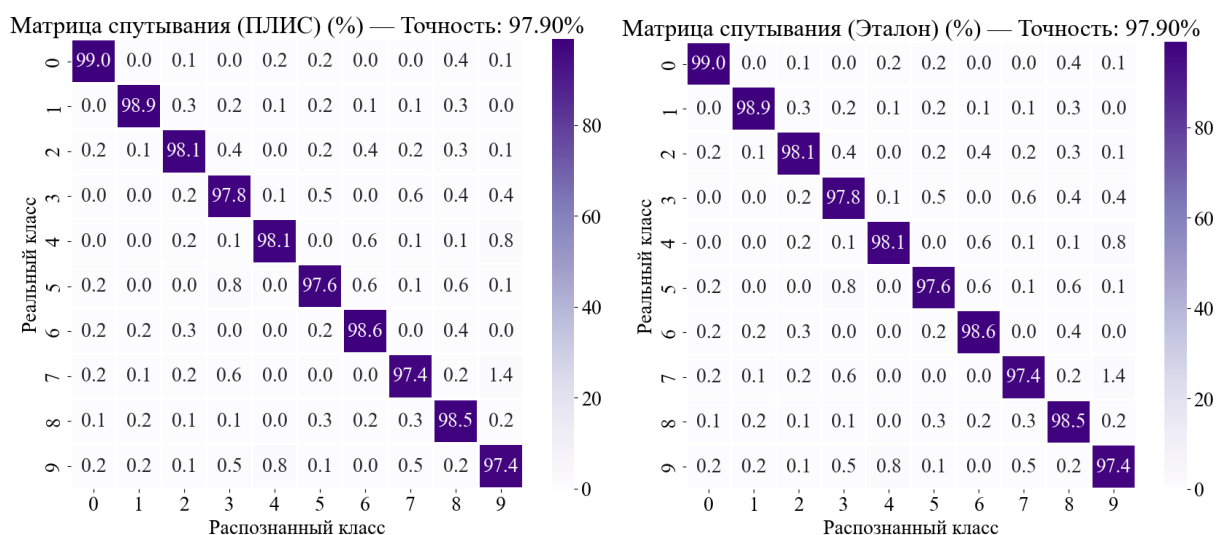


Рисунок 5.3 – Аппаратная и эталонная матрицы спутывания

Если проанализировать результаты классификации при реализации модели в фиксированной точности и с плавающей запятой, можно отметить, что точность распознавания снизилась с 98.02% до 97.9%. Это соответствует потере базовой точности на 0.12%.

Данное снижение связано с квантованием весовых коэффициентов, которое приводит к небольшому искажению параметров модели. Однако потеря точности оказалась незначительной, что свидетельствует о высокой устойчивости предложенной архитектуры к операциям квантования.

Результаты представлены в графическом материале ГУИР 431289.007 ПЛ.

5.3.2 Вторым этапом исследования стало тестирование на изображениях цифр, созданных вручную в специально разработанном консольном приложении для рисования. Программа позволяла пользователю нарисовать цифру на виртуальном холсте, сохранить изображение в формате 28×28 пикселей.

Затем отдельно запускается скрипт по обработке изображения на ПЛИС. Данный эксперимент позволил оценить устойчивость работы IP-блока к вариациям входных данных, отличающихся от стандартных изображений MNIST. Отрисовано 100 различных изображений и проверена работа данного IP-блока с построением матрицы спутывания и вычислением точности. На рисунке 5.4 представлена матрица спутывания.

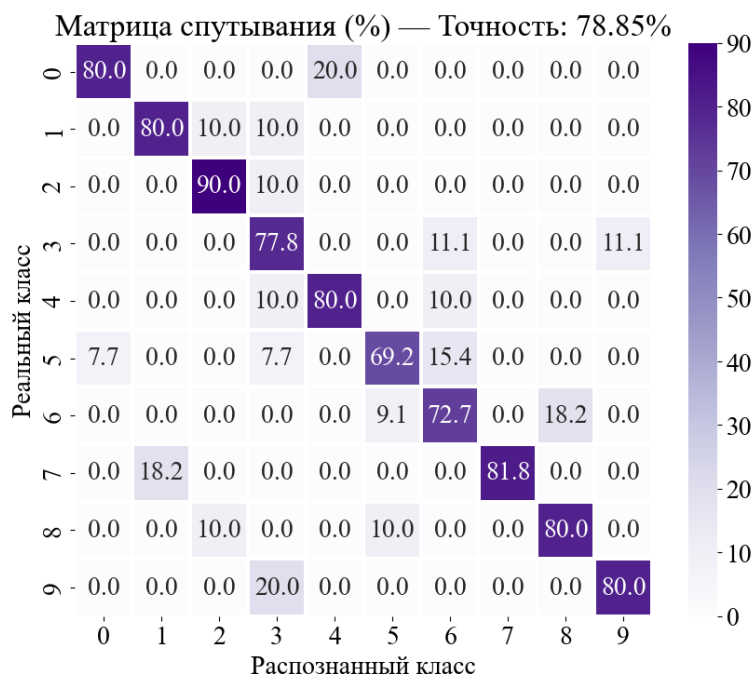


Рисунок 5.4 – Матрица спутывания на не стандартных данных

Экспериментальные результаты представлены в графическом материале ГУИР 431289.007 ПЛ.

5.3.3 Заключительный эксперимент был направлен на изучение влияния разрядности представления весовых коэффициентов на качество классификации. Для этого были проведены тестирования с различными значениями битности представления весов, такими как 16 бит, 12 бит, 8 бит и т.д.

Для повышения модульности и удобства в разработке использовался пользовательский пакет, содержащий локальные параметры, такие как разрядность данных. Это позволило централизованно управлять настройками всех модулей нейросетевой модели и обеспечило их согласованность между собой. Использование пакета упростило процесс масштабирования и модификации архитектуры, так как изменения параметров выполняются в одном месте без необходимости правки каждого отдельного модуля.

```
package nn_param_pkg;
    localparam INT_BITS = 6;
    localparam FRC_BITS = 7;
```

```

localparam BITS      = 13;
localparam PICT_SIZE = 28;
localparam WIDTH     = 784;

localparam ADDR_RC   = 5;
localparam ADDR_OUT  = 10;
localparam RCO_TYPE  = 2;

localparam NN_FSM_BUS_WIDTH = 5;
endpackage

```

На рисунке 5.5 представлен график зависимости точности от разрядности.

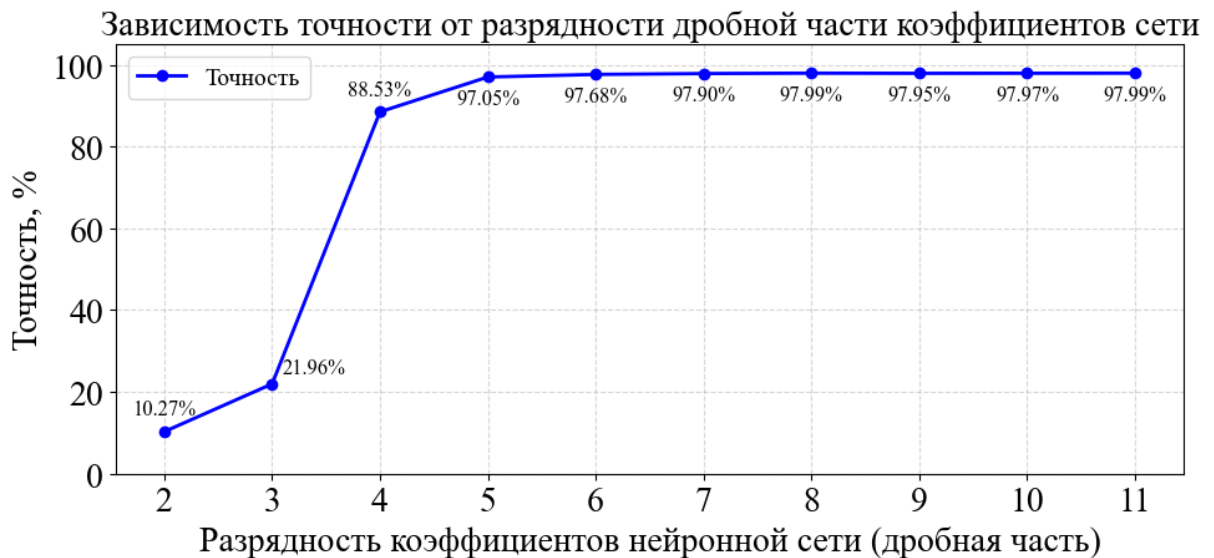


Рисунок 5.5 – Зависимость точности от разрядности

На каждом этапе тестирования сети применялись квантизованные веса с соответствующей разрядностью. Далее выполнялось тестирование на стандартной выборке MNIST, с построением матриц спутывания и вычислением точности классификации.

Анализ результатов показал, что при снижении разрядности до 8 бит потери точности классификации остаются незначительными, что подтверждает эффективность использования фиксированной арифметики для оптимизации аппаратной реализации. Однако дальнейшее уменьшение разрядности до 6 бит приводило к заметному ухудшению качества распознавания.

Результаты представлены в графическом материале ГУИР 431289.007 ПЛ.

5.4 Сравнение с аналогичными архитектурами

Для оценки эффективности предложенного IP-блока нейронной сети на базе слоя LST было проведено сравнение с рядом существующих решений,

предназначенных для задачи классификации изображений рукописных цифр из базы MNIST.

Таблица 4 – Сравнение архитектур нейронных сетей для задачи классификации рукописных цифр из базы MNIST

Автор	Архитектура нейронной сети	Число параметров	Точность
Liang, et al. (2018)[20]	784-2048-2048-10	10 100 000	98.32%
Umuroglu, et al. (2017)[21]	784-1024-1024-10	1 863 690	98.40%
Medus, et al. (2019)[16]	784-600-600-10	891 610	98.63%
Huynh[22]	784-126-126-10	115 920	98.16%
Huynh[22]	784-40-40-40-10	34 960	97.20%
Westby, et al.[23]	784-12-10	9 550	93.25%
LST-1	LST-784-10	9474	97.9%

Из таблицы видно, что большинство существующих архитектур нейронных сетей, ориентированных на высокую точность распознавания MNIST, характеризуются большим числом обучаемых параметров, что приводит к увеличению затрат ресурсов при аппаратной реализации. Например, нейронная сеть Liang et al.[20] имеет более 10 миллионов параметров, а сеть Umuroglu et al.[21] – порядка 1.8 миллиона параметров.

Предложенная архитектура LST-1 отличается принципиально меньшим числом параметров – всего 9474, что делает её крайне привлекательной для реализации в ресурсно-ограниченных средах, таких как ПЛИС или встроенные системы. При этом достигается высокая точность распознавания — 97.9%, что сравнимо с более крупными архитектурами.

Аппаратная реализация нейронной сети на базе слоя LST демонстрирует компромисс между компактностью модели и точностью классификации. Это позволяет использовать её в задачах, где критичны малое энергопотребление, быстроедействие и ограниченный объем ресурсов.

ЗАКЛЮЧЕНИЕ

В рамках данной дипломной работы была спроектирована, разработана и протестирована аппаратная реализация нейронной сети на базе программируемой логической интегральной схемы ZYBO Z7 с использованием платформы PYNQ.

На первом этапе работы была произведена формализация структуры нейронной сети в виде управляющего и операционного автоматов, что позволило эффективно спроектировать архитектуру IP-блока. Для обеспечения взаимодействия между процессорной системой и аппаратной логикой применялась среда PYNQ, обеспечивающая удобную интеграцию аппаратных модулей и программного управления.

Был разработан специализированный IP-блок, реализующий основные функции нейронной сети, включая обработку входных данных, вычисление выходов нейронов и применение функций активации. Особое внимание было уделено оптимизации аппаратной реализации: использовалась фиксированная точность представления данных для уменьшения занимаемых ресурсов без существенного ухудшения качества классификации.

Разработанная система прошла всестороннее тестирование, включающее:

- тестирование на стандартном наборе данных MNIST с построением матрицы спутывания и сравнением с эталонной моделью.

- тестирование на изображениях, созданных вручную, с целью оценки устойчивости работы.

- исследование влияния разрядности весовых коэффициентов на точность классификации.

Результаты экспериментов показали высокую точность классификации, сравнимую с полносвязными архитектурами нейронной сети, при значительно меньших затратах вычислительных ресурсов благодаря LST преобразованию.

Таким образом, поставленные цели дипломной работы были достигнуты, а все задачи успешно решены. Разработанный IP-блок нейронной сети может быть использован в составе встроенных систем реального времени, требующих высокой скорости обработки данных при ограниченных аппаратных ресурсах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Кривальцевич, Е. А. FPGA реализация нейронной сети прямого распространения для распознавания рукописных чисел / Е. А. Кривальцевич, М. И. Вашкевич // Информационные технологии и системы 2024 (ИТС 2024) = Information Technologies and Systems 2024 (ITS 2024) : материалы международной научной конференции, Минск, 20 ноября 2024 г. / Белорусский государственный университет информатики и радиоэлектроники ; редкол. : Л. Ю. Шилин [и др.]. – Минск, 2024. – С. 85–86.
- [2] Кривальцевич, Е. А. Исследование аппаратной реализации нейронной сети прямого распространения для распознавания рукописных цифр на базе FPGA = Investigation of hardware implementation of a feedforward neural network for handwritten digit recognition based on FPGA / Е. А. Кривальцевич, М. И. Вашкевич // Доклады БГУИР. – 2025. – Т. 23, № 2. – С. 101–108.
- [3] Bouvrie J. Notes on convolutional neural networks. – 2006.
- [4] Giardino D. et al. FPGA implementation of hand-written number recognition based on CNN // International Journal on Advanced Science, Engineering and Information Technology. – 2019. – Т. 9. – No. 1. – P. 167-171.
- [5] Varsamopoulos S. Neural network based decoders for the surface code. – 2019.
- [6] Agrawal V., Jagtap J., Kantipudi M. V. V. P. Decoded-ViT: A Vision Transformer Framework for Handwritten Digit String Recognition //Revue d'Intelligence Artificielle. – 2024. – Т. 38. – №. 2. – С. 523.
- [7] Agrawal V. et al. Performance analysis of hybrid deep learning framework using a vision transformer and convolutional neural network for handwritten digit recognition //MethodsX. – 2024. – Т. 12. – С. 102554.
- [8] Vashkevich M., Krivalcevic E. Compact and Efficient Neural Networks for Image Recognition Based on Learned 2D Separable Transform //2025 27th International Conference on Digital Signal Processing and its Applications (DSPA). – IEEE, 2025. – С. 1-6.
- [9] Digilent Inc. ZYBO Z7 Reference Manual, [Электронный ресурс]. – Режим доступа: <https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual>
- [10] MNIST Handwritten Numbers database [Электронный ресурс] – Электронные данные – Режим доступа: <https://yann.lecun.com/exdb/mnist/Mittal>
- [11] Xilinx, AXI Reference Guide [Электронный ресурс] – Электронные данные – Режим доступа: <https://docs.amd.com/v/u/en-US/ug1037-vivado-axi-reference-guide>, 2017.
- [12] Chu P. P. FPGA prototyping by Verilog examples: Xilinx Spartan-3 version. – John Wiley & Sons, 2011.
- [13] Tokheim, R.L. Digital Electronics: Principles and Applications. – McGraw-Hill, 2003. – 480 p.

- [14] Katz, R.H. Contemporary Logic Design. – Benjamin-Cummings, 1994. – 700 p.
- [15] Xilinx Inc. Vivado Design Suite User Guide [Электронный ресурс] – Электронные данные – Режим доступа: https://www.xilinx.com/support/documents/sw_manuals/xilinx2022_1/ug892-vivado-design-flows-overview.pdf.
- [16] Medus L. D. et al. A novel systolic parallel hardware architecture for the FPGA acceleration of feedforward neural networks //IEEE Access. – 2019. – Т. 7. – С. 76084-76103.
- [17] Xilinx, Zynq-7000 SoC Technical Reference Manual, [Электронный ресурс]. — Режим доступа: https://www.xilinx.com/support/documentation/user_guides/ug585-Zynq-7000-TRM.pdf
- [18] Li H. et al. An FPGA implementation of Bayesian inference with spiking neural networks //Frontiers in Neuroscience. – 2024. – Т. 17. – С. 1291051.
- [19] Xilinx, PYNQ: Python Productivity for Zynq, [Электронный ресурс]. — Режим доступа: <http://www.pynq.io>
- [20] S. Liang, S. Yin, L. Liu, W. Luk, and S. Wei, “FP-BNN: Binarized neural network on FPGA,” Neurocomputing, vol. 275, pp. 1072–1086, 2018
- [21] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, “FINN: A framework for fast, scalable binarized neural network inference,” in Proceedings of the 2017 ACM/SIGDA international symposium on field-programmable gate arrays, 2017, pp. 65–74.
- [22] T. V. Huynh, “Deep neural network accelerator based on FPGA,” in 4th NAFOSTED Conference on Information and Computer Science, 2017, pp. 254–257.
- [23] I. Westby, et al. “FPGA acceleration on a multilayer perceptron neural network for digit recognition,” *Journal of Supercomputing*, 2021, vol. 77, no. 12, pp. 356–373.